SLA BREACH PREVENTION AND ESCALATION SYSTEM

A PROJECT REPORT

Submitted by

RATHNA VELU P

(241401078)

in partial fulfilment of the End Semester Examination for the course

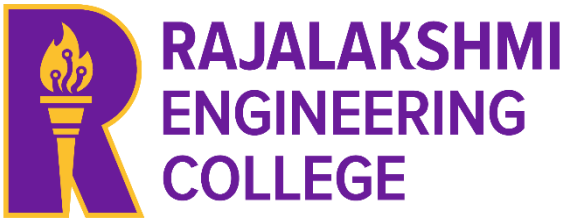
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND BUSINESS SYSTEMS



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**SLA BREACH PREVENTION AND ESCALATION SYSTEM**" is the bonafide work of **RATHNA VELU P (241401078),** who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026-2027, Semester V, under my supervision.

SIGNATURE

Dr. K. R. Sowmia, Ph.D.,
Head of the Department
Computer Science and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Mr. M. L. Alphin Ezhil Manuel
Assistant Professor
Computer Science and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

Submitted for the End Semester Examination (Individual Project) held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

I, **RATHNA VELU P (241401078),** hereby declare that the project report titled "**SLA BREACH PREVENTION AND ESCALATION SYSTEM**", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date: _____

SIGNATURE OF THE STUDENT
RATHNA VELU P
241401078

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman, **Mr. S. Meganathan, B.E., F.I.E.,,** Chairperson, **Dr. Thangam Meganathan, Ph.D.,** and Vice-Chairman, **Mr. Abhay Shankar Meganathan, B.E., M.S.,** for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, **Dr. S.N. Murugesan, M.E., Ph.D.,** Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank **Dr. K.R. Sowmia, Ph.D.,** Head of the Department, Department of Computer Science and Business Systems, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty,**Mr. M. L. Alphin Ezhil Manuel,** Assistant Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation and Zoho Creator platform for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

RATHNA VELU P

ABSTRACT

IT support teams handle service requests of varying priorities and must respond to and resolve them within defined Service Level Agreement (SLA) deadlines. Manual tracking can make it difficult to monitor deadlines, identify pending escalations, maintain request histories and communicate updates. These challenges reduce visibility into service performance and make timely escalation difficult. The SLA Breach Prevention and Escalation System was developed to organise service requests, track SLA deadlines and support escalation management through a centralised application.

The application was developed using Zoho Creator, a low-code platform for building forms, reports and workflows. It consists of four forms: Service Request, SLA Policy, Escalation Rules and Escalation Log. Reports provide views of open requests, priority-wise requests, agent-wise performance and escalation history, while a dashboard displays request and escalation summaries. Deluge scripts automate response and resolution deadline calculations, agent notifications, customer resolution emails and escalation alerts through email and Zoho Cliq. Blueprints manage the service request and escalation lifecycles. Role-based access is organised for Support Agents, Support Managers, Department Heads, Executive Leadership and the Super Admin.

The application was tested using sample service requests, blueprint transitions, email notifications and Zoho Cliq messages. Lifecycle transitions and selected notification workflows were verified. However, continuous automatic SLA monitoring and scheduled escalation generation have not been implemented. Production deployment and quantified improvements in response time, resolution time or manual effort have not been established. The project demonstrates a low-code approach to organising service requests and escalation management, providing a foundation for future enhancements such as scheduled SLA monitoring and performance measurement.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Role-Based Access Control; SLA Management.

PROJECT AT A GLANCE

Attribute	Details
Project Title	SLA Breach Prevention and Escalation System
Application Domain	IT Support / Service Request and SLA Management
Student Name & Register No	Rathna velu P - 241401078
Platform	Zoho Creator
Application Name	SLA Breach Prevention and Escalation System
Primary Forms	Service Request, SLA Policy, Escalation Rules, Escalation Log
Reports	Service Request Report, Open Service Requests, Priority-Wise Requests, Agent-Wise Performance, Escalation Log Report, Escalation Rules Report, SLA Policy Report, Escalation History
Pages	Dashboard
Automation	**Workflows**: Deadline Calculation, Agent Notification, Escalation Notification, Customer Resolution Notification, Escalation Count Update; **Blueprints**: Service Request Lifecycle, Escalation Management
Integration	Zoho Cliq, Email
User Roles	Support Agent, Support Manager, Department Head, Executive Leadership, Super Admin / CEO
Academic Year / Semester	2026-2027 / V
Course	CB23F35 – Low-Code Application Development Using Zoho Creator
Submission	October 2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 2
1.3 Objectives 2
1.4 Scope of the Project 3
1.5 Justification for a Low-Code Approach 4
1.6 Organisation of the Report 4
LITERATURE SURVEY AND SYSTEM STUDY 5
2.1 Study of the Existing System 5
2.2 Limitations of the Existing System 5
2.3 Low-Code versus Traditional Development 6
2.4 Literature Survey 6
2.5 Emerging Trends in Low-Code Development 7
2.6 Proposed System 7
REQUIREMENT ANALYSIS 9
3.1 Stakeholders and User Roles 9
3.2 Functional Requirements 9
3.3 Non-Functional Requirements 10
3.4 Hardware and Software Requirements 10
3.5 Use Case Model 11
SYSTEM DESIGN 12
4.1 System Architecture 12
4.2 Data Model 12
4.3 Form Design 13
4.4 Process and Workflow Design 14
4.5 User Interface and Navigation Design 15
4.6 Access Control Design 15
IMPLEMENTATION 17
5.1 Application Setup 17
5.2 Forms and Data Capture 17

5.3 Reports 18
5.4 Pages and Dashboards 19
5.5 Deluge Scripting 20
5.6 Workflows and Automation 21
5.7 Integrations 22
5.8 User Management and Role-Based Access 23
5.9 AI and Analytics Features 24
5.10 Pro-Code Extension 24
5.11 Mobile and Portal Access 24
TESTING AND DEBUGGING 25
6.1 Testing Strategy 25
6.2 Test Cases 25
6.3 Debugging Deluge Scripts 29
6.4 Defect Log 30
6.5 User Acceptance Testing 31
DEPLOYMENT, SECURITY AND GOVERNANCE 32
7.1 Environments and Deployment Strategy 32
7.2 Performance Optimisation 33
7.3 Security Features 34
7.4 Governance and Administration 34
RESULTS AND DISCUSSION 36
8.1 Application Output 36
8.2 Achievement of Objectives 38
8.3 Comparison with the Existing System 39
8.4 Limitations of the Low-Code Approach 40
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 40
CONCLUSION AND FUTURE ENHANCEMENTS 42
9.1 Conclusion 42
9.2 Future Enhancements 42
9.3 Learning Outcomes and Career Relevance 43
REFERENCES 44
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 45
APPENDIX B – COURSE OUTCOME MAPPING 47
APPENDIX C – COMPLETE DELUGE SOURCE CODE 48
APPENDIX D – USER MANUAL 55
D.1 Accessing the Application 55
D.2 Instructions for Support Agent 55

D.3 Instructions for Support Manager 55
D.4 Instructions for Department Head.....56
D.5 Instructions for Executive Leadership 56
D.6 Instructions for Super Admin.....56
APPENDIX E – EVALUATION RUBRIC 57
APPENDIX F – VIVA-VOCE RECORD SHEET.....59

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 6
Table 2.2: Summary of Literature Survey 6
Table 3.1: Stakeholders and User Roles 9
Table 3.2: Functional Requirements 9
Table 3.3: Non-Functional Requirements 10
Table 3.4: Hardware and Software Requirements 10
Table 3.5: Use Case Description – Manage Service Request 11
Table 4.1: Entities and Relationships 13
Table 4.2: Field Specification – Service Request Form 13
Table 4.3: Role–Permission Matrix 16
Table 5.1: Reports Implemented 18
Table 5.2: Deluge Scripts Implemented 20
Table 5.3: Workflows Implemented 21
Table 5.4: Integrations Implemented 22
Table 6.1: Test Cases 26
Table 6.2: Defect Log 30
Table 6.3: User Acceptance Feedback 31
Table 7.1: Deployment Checklist 33
Table 7.2: Performance Optimisation Results 33
Table 7.3: Security Controls Implemented 34
Table 8.1: Mapping of Objectives to Outcomes 38
Table 8.2: Existing System versus Proposed Application 39
Table A.1: Feature Coverage Checklist 45
Table B.1: Mapping of Course Outcomes to Report Chapters 47
Table E.1: Evaluation Rubric for the Individual Project 57
Table E.2: Performance Level Descriptors 57
Table F.1: Viva-Voce Questions and Responses 59
Table F.2: Marks Summary 59

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 5
Figure 3.1: Use Case Diagram of the Application 11
Figure 4.1: System Architecture of the Application 12
Figure 4.2: Entity–Relationship Diagram 12
Figure 4.3: Workflow / Process Flow for Service Request Lifecycle and Escalation Management 15
Figure 4.4: Navigation Structure and Wireframe of Dashboard 15
Figure 5.1: Application Home and Navigation Sections 17
Figure 5.2: Service Request Form in the Form Builder 18
Figure 5.3: Service Request Form in Live Mode 18
Figure 5.4: Escalation Rules Report 19
Figure 5.5: SLA Breach Prevention and Escalation Dashboard 20
Figure 5.6: Workflow Configuration 22
Figure 5.7: Zoho Cliq Connection Setup 23
Figure 5.8: Roles and Permission Profiles Configuration 23
Figure 5.9: Dashboard Analytics 24
Figure 6.1: Deluge Debug Output for the Resolution Deadline Workflow 30
Figure 7.1: Environments and Publish to Production 32
Figure 7.2: Audit Trail and Administration Settings 35
Figure 8.1: Scenario Step 1 – Service Request Created with Calculated Deadlines 36
Figure 8.2: Scenario Step 2 – Escalation Recorded and Notified 37
Figure 8.3: Scenario Step 3 – Ticket Resolved and Dashboard Updated 37
Figure 8.4: Blueprint Stages and Transitions as Configured in the Application 39
Figure 8.5: Application Components on the No-Code, Low-Code and Pro-Code Spectrum 41

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
JSON	JavaScript Object Notation
LCAP	Low-Code Application Platform
OAuth	Open Authorization
RBAC	Role-Based Access Control
REST	Representational State Transfer
SLA	Service Level Agreement
SDLC	Software Development Life Cycle
SSO	Single Sign-On
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
Zia	Zoho’s AI assistant
Zoho Cliq	Zoho's team communication application

CHAPTER 1
INTRODUCTION

1.1 Background

Information Technology (IT) support is an important function in organisations that depend on computer systems, software applications and network services for their daily operations. Employees rely on IT support teams to resolve technical issues such as network connectivity problems, software errors, hardware failures and access-related requests. These requests must be handled within defined Service Level Agreement (SLA) targets to maintain service quality and minimise disruption.

In a typical IT support environment, service requests are received from users and assigned to support agents based on their priority and category. Each request requires details such as the request description, customer information, assigned agent, priority and current status to be recorded. The support team must monitor the time taken to provide the first response and resolve the request. Requests that approach or exceed their SLA deadlines may require escalation to higher-level support personnel or management.

The proposed project is designed for a representative IT support department involving Support Agents, Support Managers, Department Heads and Executive Leadership. Support Agents handle service requests, while Support Managers oversee request handling and escalation activities. Department Heads manage SLA policies and escalation rules, and Executive Leadership requires visibility into service operations. The Super Admin manages the application and its access settings.

The SLA Breach Prevention and Escalation System provides a centralised application for recording service requests, maintaining SLA policies, defining escalation rules and tracking escalation activities. Developed using Zoho Creator, the application includes forms, reports, a dashboard, Deluge scripts and workflow automation. It calculates response and resolution deadlines based on the selected SLA policy and supports notifications through email and Zoho Cliq. Blueprints are used to manage service request and escalation lifecycles. The application aims to improve the organisation and visibility of IT support activities and provide a structured way to manage SLA-related information.

1.2 Problem Statement

Managing IT service requests requires support teams to record requests, monitor SLA deadlines, assign responsibilities and communicate escalation events. When these activities are handled through manual tracking or separate records, it becomes difficult to maintain consistent information and monitor the progress of requests. The project addresses the need for a centralised system that organises service request data, supports deadline calculations and provides structured escalation management.

The major problems identified are:

- Difficulty in tracking response and resolution deadlines for multiple service requests.
- Lack of a centralised record for SLA policies, escalation rules and escalation history.
- Delays in communicating ticket assignments and escalation events to responsible personnel.
- Limited visibility into request status, priority-wise workload and agent-wise performance.
- Need for structured access permissions for different support and management roles.

1.3 Objectives

- To design and develop forms for managing service requests, SLA policies, escalation rules and escalation logs.
- To calculate response and resolution deadlines using Deluge scripting and workflows.
- To provide reports and a dashboard for monitoring service requests and escalation activities.
- To integrate Zoho Creator with Zoho Cliq and email notifications for communicating assignment and escalation events.
- To implement role-based access for support and management personnel.
- To manage service request and escalation lifecycles using Zoho Creator Blueprints.

1.4 Scope of the Project

The project focuses on developing an IT service request and SLA management application using Zoho Creator. Its scope includes:

In-scope activities:

- Recording and managing IT service requests.
- Maintaining SLA policies for different priority levels.
- Defining escalation rules and recording escalation events.
- Calculating response and resolution deadlines.
- Sending assignment, escalation and resolution notifications through configured email workflows and Zoho Cliq.
- Managing service request and escalation stages using Blueprints.
- Providing reports and a dashboard for monitoring requests and escalation records.
- Configuring profiles and data-sharing rules for Support Agents, Support Managers, Department Heads, Executive Leadership and the Super Admin.

Out-of-scope activities:

- Continuous scheduled monitoring of all SLA deadlines and automatic creation of escalation records.
- AI-based prediction of SLA breaches.
- Integration with external CRM or third-party ticketing platforms.
- A dedicated customer self-service portal.
- Offline mobile operation.
- Verified production deployment and measured performance improvements.

The application is developed on Zoho Creator and is intended to support its users through the platform's available application access. Features not implemented in the current version are considered possible future enhancements.

1.5 Justification for a Low-Code Approach

A low-code approach was selected because the project requires forms, reports, workflows, notifications and access controls that can be developed using visual tools with limited conventional programming. Zoho Creator provides drag-and-drop form building, report generation, workflow automation and Deluge scripting, making it suitable for developing an IT support management application [1].

The platform allows the application to be modified as business requirements change. Forms and reports can be updated through its visual interface, while Deluge scripts provide additional control over deadline calculations and notification processes. Zoho Creator also supports integrations with other applications, role-based access controls and access through web and mobile devices [1], [2].

Another advantage is that Zoho Creator provides cloud hosting and manages much of the underlying infrastructure, reducing the need for separate server setup and maintenance [2]. These capabilities make it appropriate for this project, which requires a centralised and maintainable solution for service request and escalation management without developing every component from the beginning.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In a typical IT support environment, service requests are received from employees and recorded manually or in spreadsheets. The support team reviews each request, assigns it to an agent and monitors its progress. SLA deadlines are checked manually, and escalation details are communicated through email or other messaging channels. Reports are prepared by reviewing the available records.

The general process is as follows:

1. A user reports an IT issue.
2. Support personnel record the request and its priority.
3. The request is assigned to a support agent.
4. The agent works on the request and updates its status.
5. The support team monitors the response and resolution deadlines.
6. Delayed requests are communicated to the appropriate manager.
7. The request is resolved and its details are retained for reference.

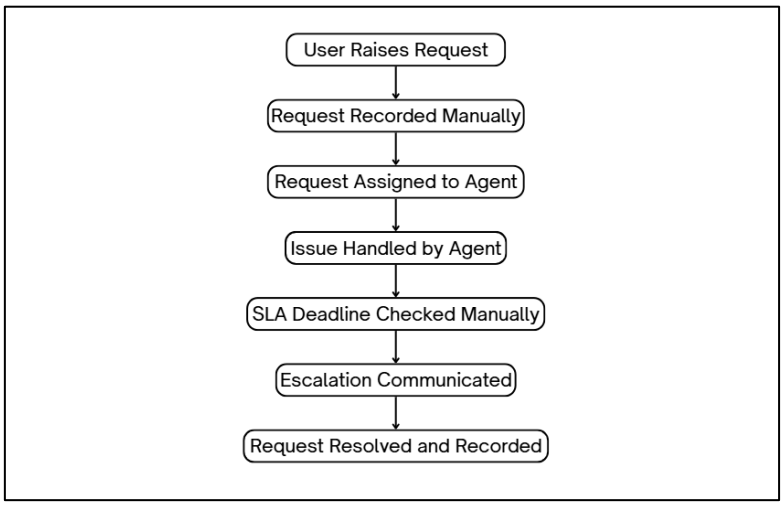


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- Manual tracking makes SLA deadline monitoring difficult.
- Request, policy and escalation details may be stored separately.

- Delayed communication can affect timely escalation.
- Preparing reports requires manual effort.
- Access to request information may not be properly organised by role.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	8-10 weeks	2-3 weeks
Skills required	Programming, database and web development skills.	Visual development and basic Deluge scripting skills.
Hosting & infrastructure	Requires separate hosting and infrastructure management.	Cloud platform hosting is provided.
Mobile application	Requires additional mobile development.	Supports access through Zoho Creator mobile applications.
Security & access control	Must be developed and configured separately.	Provides built-in user, profile and permission settings.
Maintenance & change	Changes may require code modifications.	Forms, reports and workflows can be modified visually.
Customisation limits	Greater flexibility for specialised requirements.	Customisation depends on platform features and Deluge capabilities.
Cost	May involve higher development and infrastructure costs.	Subscription-based platform; cost depends on the selected plan.

2.4 Literature Survey

The following studies and official documentation provide the background for low-code development, workflow automation and IT service management.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Bucaioni et al., 2022	Modelling in Low-Code Development	Low-code supports visual modelling, automation and rapid application development [3].	Supports the use of Zoho Creator for application development.
2	Bock and Frank, 2021	Low-Code Platform	Discusses low-code platforms and their role in organisational development [4].	Provides the basis for selecting a low-code platform.

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
3	Pinho et al., 2023	Usability in Low-Code Platforms	Reviews usability aspects and challenges of low-code platforms [5].	Highlights the importance of simple forms and navigation.
4	Zoho Corporation, 2026	Zoho Creator Documentation	Describes forms, workflows, Deluge scripting, users and application deployment [6].	Supports the implementation of application modules and automation.
5	UK Government, 2015	Service Management Good Practice	Explains incident management and handling service requests according to SLAs [7].	Supports the project's SLA tracking and escalation requirements.

2.5 Emerging Trends in Low-Code Development

Low-code development is evolving through several important trends:

- AI-assisted development: AI tools help developers create and improve applications more efficiently.
- Citizen development: Business users can build applications using visual development tools.
- Hyper-automation: Workflows and integrations reduce repetitive manual activities.
- Composable applications: Reusable components and integrations help build flexible business solutions.

These trends influenced the project through visual form development, Deluge-based automation, Zoho Cliq notifications and role-based access configuration. AI-based prediction and advanced automation are potential future enhancements rather than implemented features.

2.6 Proposed System

The proposed system is a centralised IT support application developed using Zoho Creator. It consists of four forms: Service Request, SLA Policy, Escalation Rules

and Escalation Log. Reports and a dashboard provide visibility into request status and escalation activities.

Deluge workflows calculate response and resolution deadlines and send assignment, escalation and resolution notifications. Zoho Cliq is used for escalation and assignment communication. Blueprints manage service request and escalation stages, while role-based permissions organise access for support and management users.

The proposed system addresses the limitations of manual tracking by maintaining centralised records, supporting deadline calculations, providing structured escalation records and offering reports for monitoring. Continuous scheduled SLA monitoring and automatic escalation generation remain future enhancements.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Super Admin	Application administrator; organisationally treated as CEO.	Manage users, application settings and all records.
Executive Leadership	Senior management.	View and edit relevant operational information.
Department Head	Oversees support department policies.	Manage SLA Policies and Escalation Rules.
Support Manager	Supervises support operations.	Monitor requests and manage escalation records.
Support Agent	Handles IT service requests.	Create, update and resolve assigned requests.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow users to create and maintain service request records.	Support Agent, Support Manager	High
FR-02	The system shall calculate response and resolution deadlines using the selected SLA Policy.	System	High
FR-03	The system shall send an email notification when a request is assigned to an agent.	System, Support Agent	Medium
FR-04	The system shall allow users to create and maintain SLA Policies and Escalation Rules.	Department Head	High
FR-05	The system shall allow users to record and track escalation events.	Support Manager	High
FR-06	The system shall send email and Zoho Cliq notifications when an escalation record is created, subject to the configured notification rule.	System, Support Manager	Medium
FR-07	The system shall notify customers by email when a service request is resolved.	System, Support Agent	Medium
FR-08	The system shall update escalation counts when an escalation log is created.	System	High

FR ID	Requirement	Role(s)	Priority
FR-09	The system shall display reports and a dashboard containing service request and escalation information.	Support Manager, Department Head, Executive Leadership	Medium
FR-10	The system shall support service request and escalation lifecycle transitions through Blueprints.	Support Agent, Support Manager	High
FR-11	The system shall provide role-based access through configured profiles and data-sharing rules.	Super Admin	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Forms and reports should load within a reasonable time under normal usage.
NFR-02	Security	Users shall access application functions and records according to configured permissions.
NFR-03	Usability	The application shall provide clear forms, reports and navigation through the Zoho Creator interface.
NFR-04	Availability	The application shall provide clear forms, reports and navigation through the Zoho Creator interface.
NFR-05	Maintainability	Forms, workflows, reports and scripts should be modifiable when requirements change.
NFR-06	Auditability	Service request and escalation information shall be maintained for tracking and review.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge
Integration service	Zoho Cliq
Notification service	Email through Zoho Creator
Mobile	Zoho Creator mobile app, if used
Client hardware	Computer with internet access and a modern web browser

3.5 Use Case Model

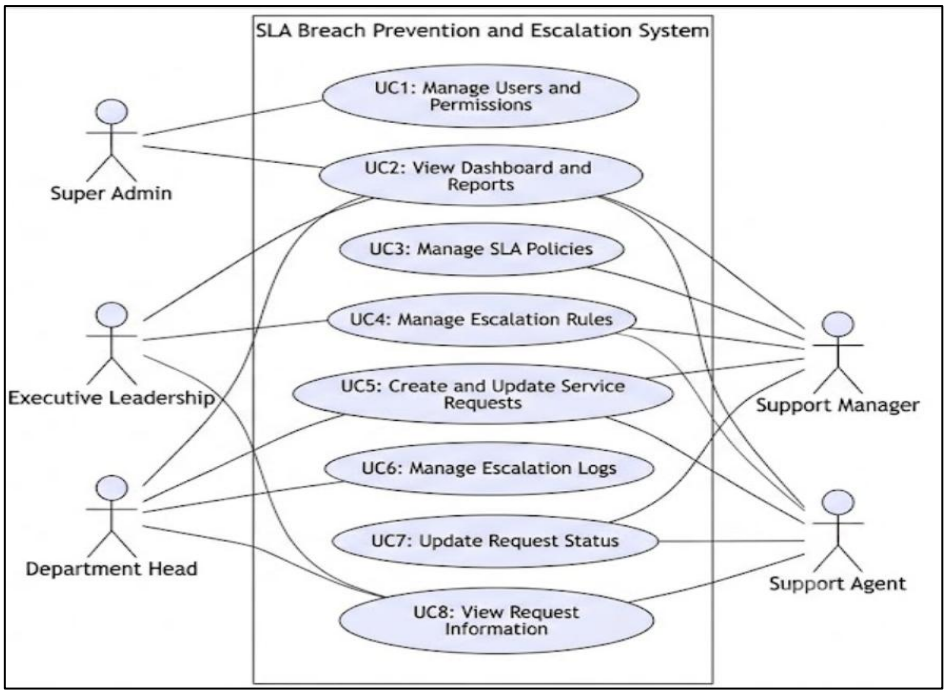


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Manage Service Request

Use Case ID	UC – 01
Use Case Name	Manage Service Request
Actor(s)	Support Agent, Support Manager
Pre-condition	User is logged in with appropriate permissions.
Main Flow	1. Open Service Request form. 2. Enter request details. 3. Select priority and SLA Policy. 4. Save the record. 5. System calculates deadlines.
Alternate Flow	Mandatory fields are missing; the record cannot be submitted.
Post-condition	Service request is stored with calculated deadlines.
Related FR	FR-01, FR-02

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

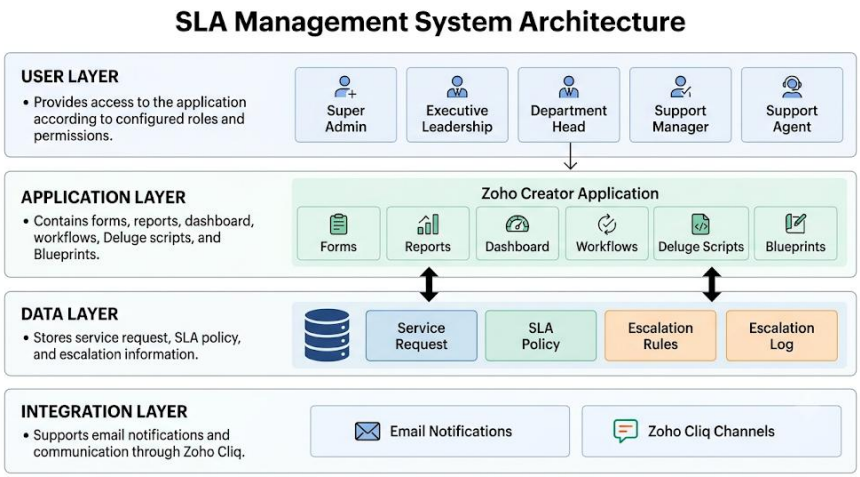


Figure 4.1: System Architecture of the Application

The architecture consists of four layers:

1. **User Layer**: Provides access to the application according to configured roles and permissions.
2. **Application Layer**: Contains forms, reports, dashboard, workflows, Deluge scripts and Blueprints.
3. **Data Layer**: Stores service request, SLA policy and escalation information.
4. **Integration Layer**: Supports email notifications and communication through Zoho Cliq.

4.2 Data Model

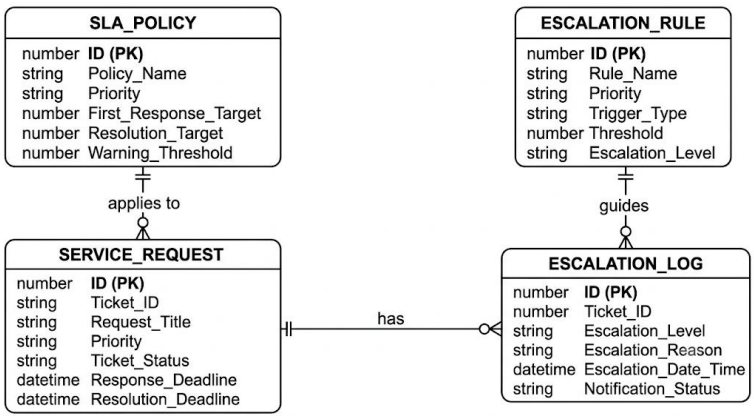


Figure 4.2: Entity–Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
SLA Policy	Policy Name, Priority, Response Target, Resolution Target	Service Request	One-to-Many	Lookup in Service Request
Service Request	Ticket ID, Request Title, Priority, Status, Deadlines	SLA Policy, Escalation Log	Many-to-One / One-to-Many	Lookup fields
Escalation Rules	Rule Name, Priority, Trigger Type, Threshold	Escalation Log	Rule-based association	Deluge lookup
Escalation Log	Related Service Request, Escalation Level, Reason, Status	Service Request, Escalation Rules	Many-to-One / Rule-based	Lookup and Deluge script

4.3 Form Design

The application contains four forms. Each form captures information required for service request handling, SLA configuration and escalation management. The following tables summarise the main fields and their specifications.

Table 4.2: Field Specification – Service Request Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Ticket ID	ticket_id	Auto Number	Yes	System generated	Unique ticket number
Request Title	request_title	Single Line	Yes	-	Request subject
Description	description	Multi Line	Yes	-	Issue details
Customer Name	customer_name	Name	Yes	-	Requester
Customer Email	customer_email	Email	Yes	Valid email	Requester contact
Category	category	Drop Down	Yes	Network, Access, Hardware, Software, Other	Issue category

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Priority	priority	Drop Down	Yes	Critical, High, Medium, Low	Request priority
Assigned Agent	assigned_agent	User / Lookup	Yes	Select agent	Responsible agent
Creation Date and Time	creation_date_time	Date-Time	Yes	Current date and time	Request creation
SLA Policy	sla_policy	Lookup	Yes	Select policy	Related SLA Policy
First Response Time	first_response_time	Number	No	-	Minutes
Resolution Time	resolution_time	Number	No	-	Minutes
Ticket Status	ticket_status	Drop Down	Yes	Open, In Progress, Resolved, Closed	Request lifecycle
SLA Status	sla_status	Drop Down	No	On Track, Resolved Within SLA, Resolved Breached	SLA result
Escalation Level	escalation_level	Number	No	-	Current escalation level
Escalation Count	escalation_count	Number	No	Default: 0	Updated through workflow

4.4 Process and Workflow Design

The workflows calculate deadlines and send notifications based on configured events. The Blueprints control the stages and transitions of service requests and escalation records. Continuous scheduled SLA monitoring and automatic escalation creation are not included in the current implementation.



Figure 4.3: Workflow / Process Flow for Service Request Lifecycle and Escalation Management

4.5 User Interface and Navigation Design

The application provides navigation through its forms, reports and dashboard. Users access the available components according to their permissions.

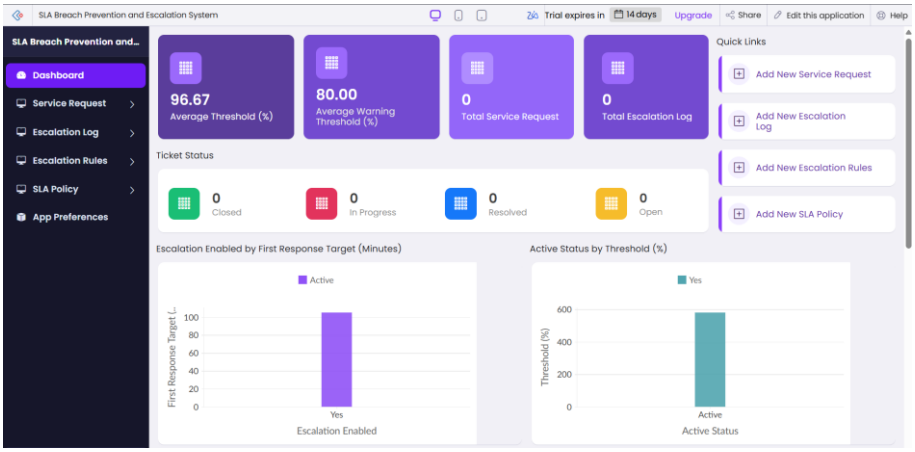


Figure 4.4: Navigation Structure and Wireframe of Dashboard

4.6 Access Control Design

The application uses role-based access control through Zoho Creator profiles and data-sharing rules. The Super Admin has full administrative access, while other roles

receive permissions according to their responsibilities. Delete permissions are restricted to the Super Admin.

Table 4.3: Role–Permission Matrix

Legend: A – Add; E – Edit; V – View; D – Delete; — – No access.

Component	Super Admin	Executive Leadership	Department Head	Support Manager	Support Agent
Service Request	A E V D	V E	V	A E V	A E V
SLA Policy	A E V D	V E	A E V	V	-
Escalation Rules	A E V D	V E	A E V	V	-
Escalation Log	A E V D	V E	V	A E V	A E V
Reports	V	V	V	V	V
Dashboard	V	V	V	V	-
User Management	A E V D	-	-	-	-

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The SLA Breach Prevention and Escalation System was developed using Zoho Creator. The application contains four main forms: Service Request, SLA Policy, Escalation Rules and Escalation Log. Reports and a dashboard provide access to ticket information, SLA configurations and escalation records.

The application was configured with user profiles and data-sharing rules to support different levels of access. Deluge scripts, form workflows and Blueprints were used to automate deadline calculation, notifications and request status management.

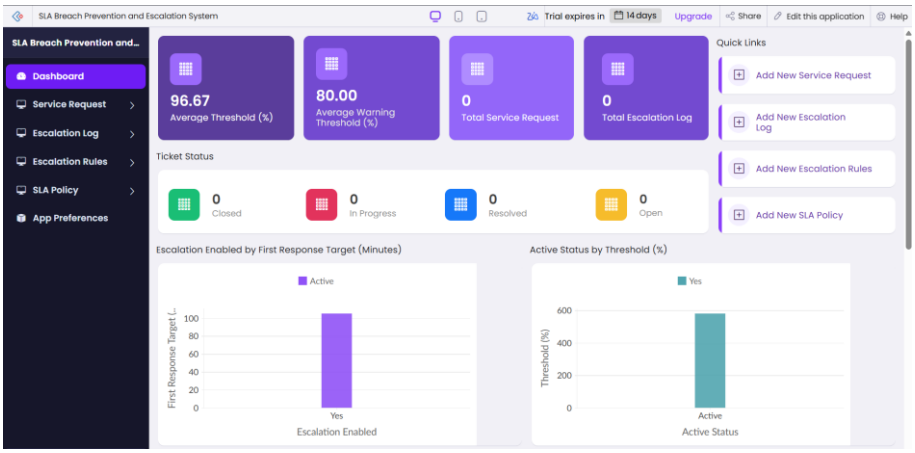


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

Four forms were created to store and manage service request and escalation information.

- **Service Request**: Captures ticket details, customer information, priority, assigned agent, SLA policy, deadlines and ticket status.
- **SLA Policy**: Stores first-response targets, resolution targets and warning thresholds for different priorities.
- **Escalation Rules**: Defines escalation levels, trigger types, thresholds and notification settings.
- **Escalation Log**: Records escalation events, related tickets, notification status and actions taken.

The Service Request form uses a lookup field to connect each ticket to an SLA Policy. The Escalation Log form uses a lookup field to associate an escalation with its Service Request. Drop-down fields maintain consistent values for priority, category, ticket status and escalation status.

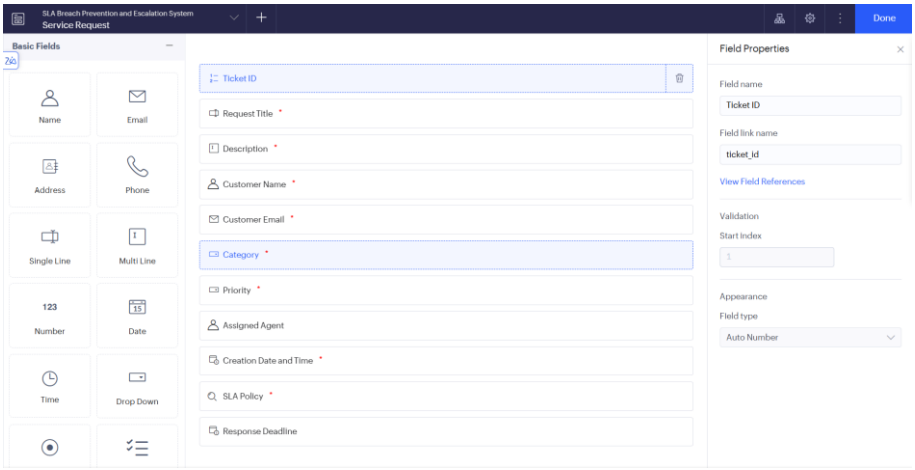


Figure 5.2: Service Request Form in the Form Builder

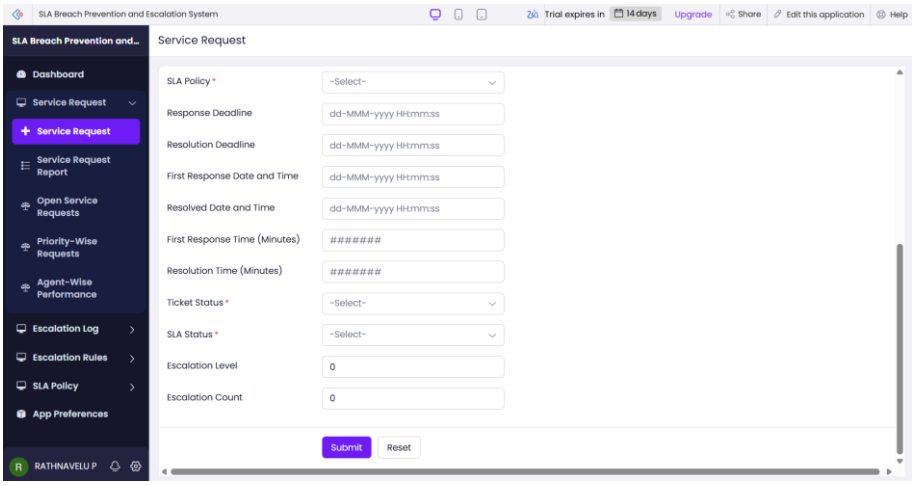


Figure 5.3: Service Request Form in Live Mode

5.3 Reports

The application provides reports for monitoring service requests, agent performance, SLA policies and escalation activities. These reports help users view records according to their operational requirements.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Purpose
Service Request Report	List	Service Request	Displays service request records

Report Name	Type	Source Form	Purpose
Open Service Requests	List	Service Request	Shows open tickets
Priority-Wise Requests	List	Service Request	Displays tickets by priority
Agent-Wise Performance	List	Service Request	Provides agent-related ticket information
Escalation Log Report	List	Escalation Log	Displays escalation records
Escalation Rules Report	List	Escalation Rules	Displays configured escalation rules
SLA Policy Report	List	SLA Policy	Displays SLA policy details
Escalation History	List	Escalation Log	Provides escalation history

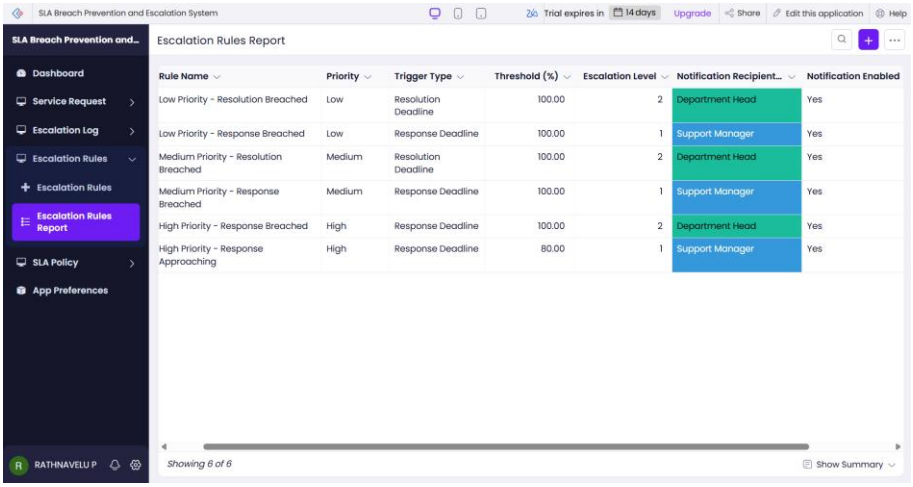


Figure 5.4: Escalation Rules Report

5.4 Pages and Dashboards

The application includes a dashboard that provides an overview of service requests and escalation activities. It displays the total number of service requests, total escalation logs, ticket status counts, average escalation-rule threshold and average SLA warning threshold.

Quick links are provided to add a Service Request, Escalation Log and Escalation Rule. These components help users access frequently used functions from one place.

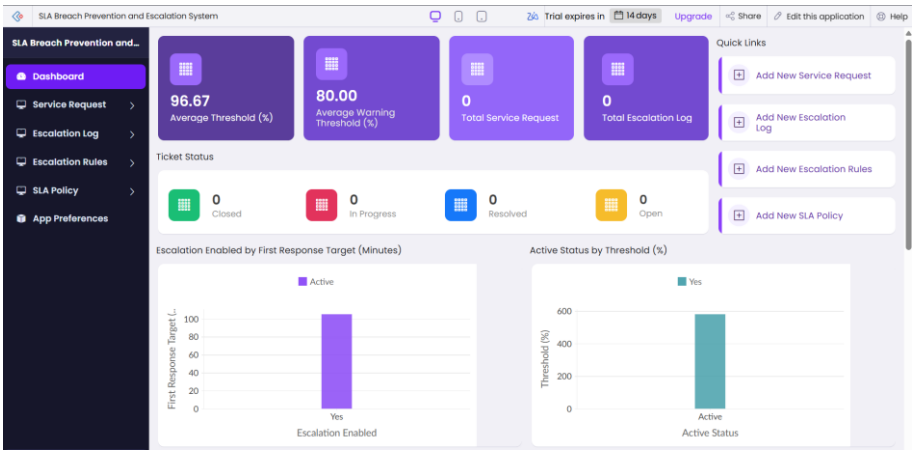


Figure 5.5: SLA Breach Prevention and Escalation Dashboard

5.5 Deluge Scripting

Deluge scripting was used to automate important operations in the application. The scripts retrieve related records, calculate deadlines, send notifications and update escalation information.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Service Request – On Add / Success	Calculate response deadline	Record lookup, date-time calculation
DS-02	Service Request – On Add / Success	Calculate resolution deadline	Record lookup, date-time calculation
DS-03	Service Request – On Add / Success	Notify assigned agent	Conditions, email and Cliq notification
DS-04	Escalation Log – On Add / Success	Notify about escalation events	Record lookup, conditions, email and Cliq
DS-05	Service Request – On Edit / Success	Notify customer on resolution	Status condition, email notification
DS-06	Escalation Log – On Add / Success	Increment escalation count	Record lookup, null handling, field update

Code 5.1: Response Deadline Calculation

```
// DS-01 | Form: Service request | Event: On Add / Success
// Purpose: Calculate response deadline
policyRec = SLA_Policy[ID == input.sla_policy.ID];
if(policyRec != null)
{
   input.response_deadline =
      zoho.currenttime.addMinutes(policyRec.first_response_target);
}
```

Code 5.2: Escalation Count Update

```
// DS-02 | Form: Escalation log | Event: On Add / Success
// Purpose: Increment escalation count
serviceRequestRec = service_request[ID == input.service_request.ID];
if(serviceRequestRec != null)
{
   currentCount = ifnull(serviceRequestRec.escalation_count, 0);
   serviceRequestRec.escalation_count = currentCount + 1;
   serviceRequestRec.escalation_level = input.escalation_level;
}
```

The scripts automate deadline calculations and notification activities. The escalation count script updates the related service request whenever an escalation log is added. The complete scripts are provided in Appendix C.

5.6 Workflows and Automation

Form workflows automate actions when records are added or edited. Blueprints manage the lifecycle of service requests and escalation records.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Action(s)
Calculate Response Deadline on Creation	Form Workflow	Service Request – On Add / Success	Calculates response deadline using SLA Policy
Calculate Resolution Deadline on Creation	Form Workflow	Service Request – On Add / Success	Calculates resolution deadline
Notify Agent on Ticket Assignment	Form Workflow	Service Request – On Add / Success	Sends email and Cliq notification
Notify on Escalation Event	Form Workflow	Escalation Log – On Add / Success	Sends escalation email and Cliq notification
Notify Customer on Ticket Resolution	Form Workflow	Service Request – On Edit / Success	Sends customer email when ticket is resolved
Service Request Lifecycle	Blueprint	Ticket stage transitions	Manages Open, In Progress, Resolved and Closed stages
Escalation Management	Blueprint	Escalation stage transitions	Manages escalation progress and resolution

The Service Request Blueprint includes transitions for assigning, resolving, closing and reopening tickets. The Escalation Management Blueprint supports escalation, further escalation and resolution.

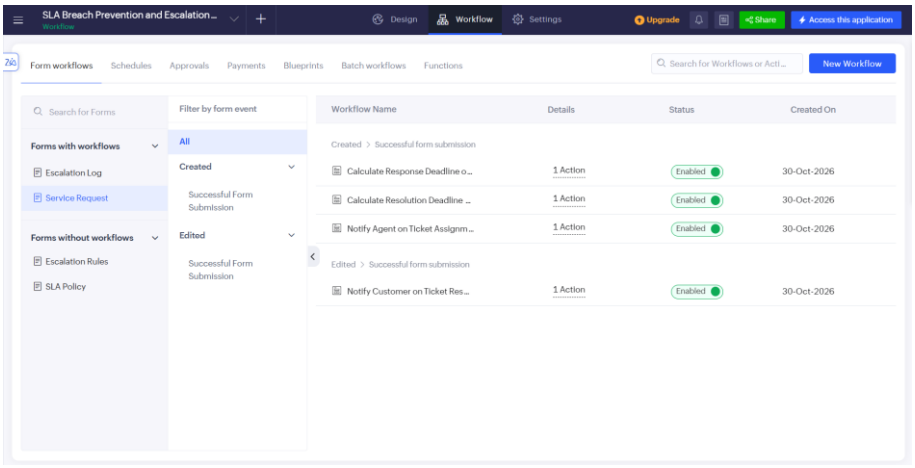


Figure 5.6: Workflow Configuration

5.7 Integrations

The application integrates with Zoho Cliq to send notifications to designated channels. A Zoho connection was configured and authorised to post messages to Cliq channels.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Zoho Cliq – Request Notifications	`itsupportrequests` channel	Deluge Cliq task using connection	OAuth connection	Sends ticket assignment notifications
Zoho Cliq – Escalation Notifications	`itsupportescalations` channel	Deluge Cliq task using connection	OAuth connection	Sends escalation notifications

Code 5.3: Zoho Cliq Notification

```
zoho.cliq.postToChannel(
   "itsupportescalations",
   "Escalation notification for Ticket ID: " + input.ticket_id,
   "cliq_escalation_connection"
);
```

The Cliq integration was tested by posting an escalation message to the escalation channel. The message displayed the ticket ID, request title, priority,

escalation level, reason and action taken. Email notifications were also tested for ticket assignment, escalation and customer resolution.

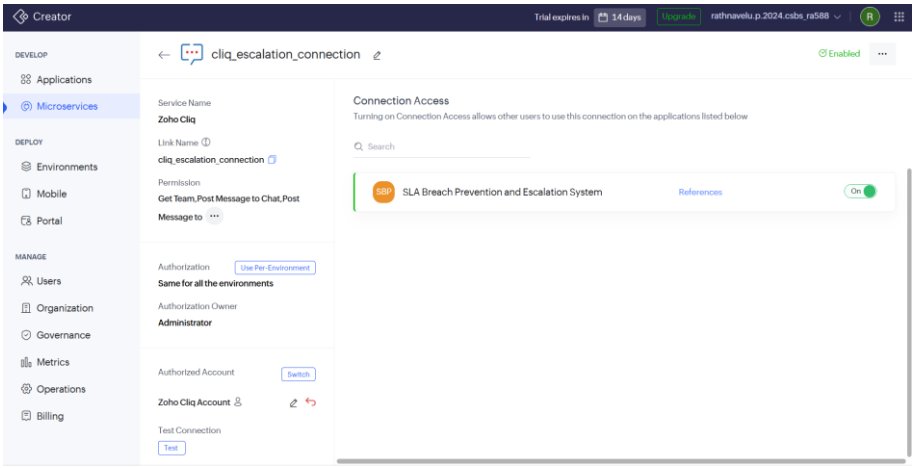


Figure 5.7: Zoho Cliq Connection Setup

5.8 User Management and Role-Based Access

User management was configured using Zoho Creator profiles and data-sharing rules. The application follows an organisational hierarchy consisting of Super Admin, Executive Leadership, Department Head, Support Manager and Support Agent.

Seven user accounts were configured, including the Super Admin account, three Support Agents, one Support Manager, one Department Head and one Executive Leadership user.

Profiles control access to application components, while data-sharing rules define how records are shared between organisational roles. Delete permissions are restricted to the Super Admin. Executive Leadership has View and Edit permissions as configured.

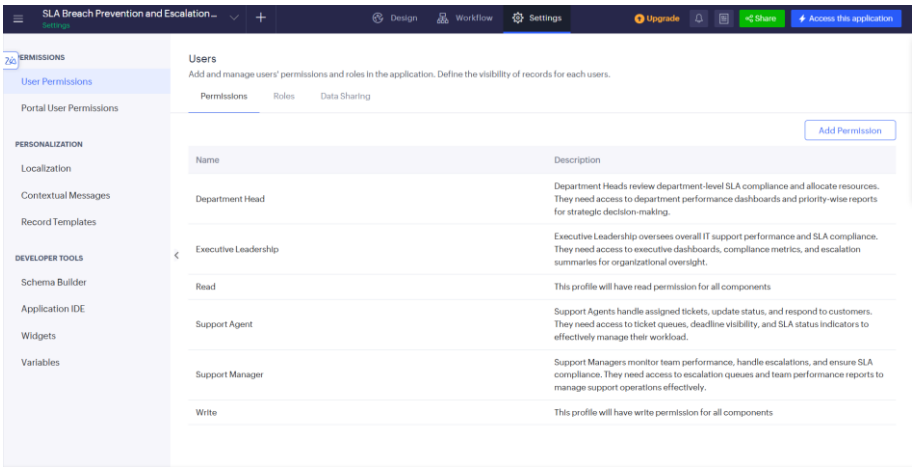


Figure 5.8: Roles and Permission Profiles Configuration

5.9 AI and Analytics Features

AI-based features such as Zia predictions, OCR and sentiment analysis were not implemented in the current application. The dashboard provides basic analytical information through ticket counts and average threshold values. Reports also support the review of priority-wise requests, agent-wise performance and escalation records.

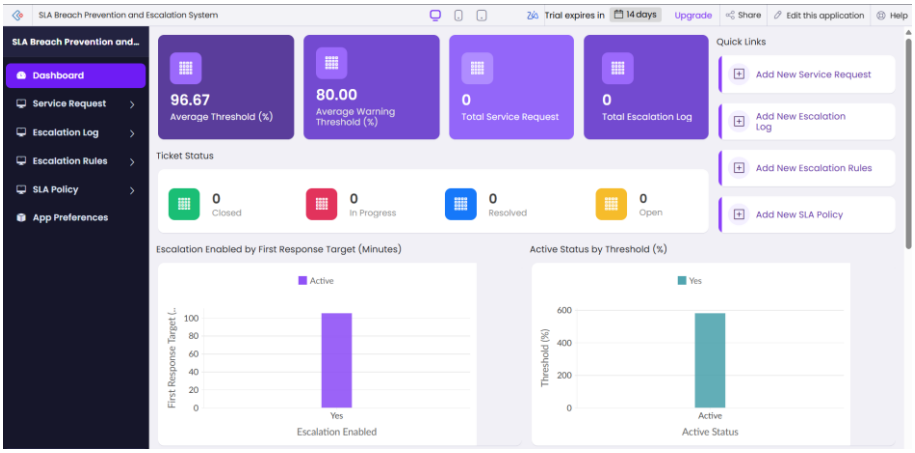


Figure 5.9: Dashboard Analytics

5.10 Pro-Code Extension

The application was developed using Zoho Creator's low-code features and Deluge scripting. Advanced pro-code extensions such as Zoho Catalyst, custom JavaScript widgets and external application development were not used. These technologies may be considered in future versions if more advanced processing or custom interfaces are required.

5.11 Mobile and Portal Access

The application was developed primarily for use through Zoho Creator. A separate customer portal and offline mobile functionality were not implemented in the current version. Mobile access through the Zoho Creator mobile application may be considered for future enhancement.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was carried out to check the forms, Deluge workflows, Blueprints, reports, dashboard, notifications and access configuration of the SLA Breach Prevention and Escalation System. The application was examined in the Zoho Creator environment using sample service requests and escalation records.

The following testing levels were considered:

1. **Form testing**: The Service Request, SLA Policy, Escalation Rules and Escalation Log forms were checked for field configuration, data entry and record creation.
2. **Workflow testing**: The deadline calculation workflows, agent assignment notification, escalation notification, customer resolution notification and escalation-count update were examined.
3. **Blueprint testing**: The Service Request Lifecycle and Escalation Management Blueprints were tested using their configured transitions.
4. **Integration testing**: Zoho Cliq notifications were checked using the configured connection and channel names. Email notifications were also checked.
5. **Role and permission testing**: The configured profiles and data-sharing rules were reviewed. Complete login-based testing of every role should be recorded separately where performed.
6. **User acceptance testing**: Feedback from users in their assigned roles should be collected and entered in Table 6.3.

The test results below distinguish features that were verified from tests for which a separate execution result has not been recorded.

6.2 Test Cases

Table 6.1 lists 26 test cases covering every functional requirement in Table 3.2 (FR-01 to FR-11). They include positive, negative, workflow, integration and role-access tests; the type is shown in brackets in the Test Description column. The test cases were derived from exported application configuration and the scripts in Appendix C.

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	(Positive) Create a service request	Enter all required fields and submit the Service Request form.	Record is created successfully.	Service Request form and sample records are available.	Pass
TC-02	FR-03	(Positive) Notify assigned agent	Create a request with an assigned agent.	Agent receives an assignment notification.	Assignment email was tested and received.	Pass
TC-03	FR-07	(Positive) Notify customer on resolution	Change a request status to Resolved.	Customer receives a resolution email.	Resolution email was tested and received.	Pass
TC-04	FR-09	(Positive) View dashboard indicators	Open the application dashboard.	Request totals, escalation totals and status counts are displayed.	Dashboard contains the configured indicators.	Pass
TC-05	FR-09	(Positive) Open service request reports	Open the Service Request and related reports.	Relevant request records are displayed.	Service Request reports are present.	Pass
TC-06	FR-01	(Negative) Submit with mandatory fields empty	Open the Service Request form, leave the mandatory fields empty and click Submit.	Form is not submitted; mandatory-field messages appear; no record is created.	Submission was blocked and mandatory-field validation messages were displayed. No record was created.	Pass
TC-07	FR-01	(Negative) Invalid customer email	Enter text without "@" in Customer Email and submit.	Email validation rejects the value; record is not created.	Invalid email input was rejected, and the form was not submitted.	Pass
TC-08	FR-02	(Workflow) Deadlines on creation	Create a ticket with an SLA Policy of known First Response Target and Resolution Target (minutes).	Response Deadline = processing time + First Response Target; Resolution Deadline = Creation Date and Time + Resolution Target.	Response and resolution deadlines were calculated correctly using the selected SLA Policy.	Pass
TC-09	FR-02	(Negative) No SLA Policy selected	Submit a ticket without selecting an SLA Policy.	Submission is blocked because SLA Policy is mandatory; no deadlines are calculated.	Submission without an SLA Policy was blocked. No record was created.	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-10	FR-03	(Negative) No agent assigned	Create a ticket leaving Assigned Agent empty.	Ticket is saved; no assignment email or Cliq message is sent; no error is shown.	The ticket was saved without an assigned agent, and no assignment notification was sent.	Pass
TC-11	FR-04	(Positive) Maintain SLA Policy and Escalation Rule	As Department Head, add an SLA Policy and an Escalation Rule with all mandatory fields; open their reports.	Both records are saved; Warning Threshold defaults to 80%; records appear in SLA Policy Report and Escalation Rules Report.	SLA Policy and Escalation Rule records were created successfully and appeared in their respective reports.	Pass
TC-12	FR-04	(Negative) Duplicate policy or rule name	Add an SLA Policy with an existing Policy Name; repeat with an existing Rule Name.	Duplicate is rejected; no record is created.	Duplicate Policy Name and Rule Name entries were rejected.	Pass
TC-13	FR-05	(Positive) Record an escalation	As Support Manager, add an Escalation Log for an existing ticket with all mandatory fields.	Record is saved and appears in Escalation Log Report and Escalation History.	The Escalation Log was saved and appeared in the Escalation Log Report and Escalation History.	Pass
TC-14	FR-05	(Negative) Incomplete escalation record	Submit an Escalation Log without Related Service Request, Escalation Reason or Resolution Status.	Submission is blocked by mandatory-field messages.	Submission was blocked because the required escalation fields were missing	Pass
TC-15	FR-06	(Integration) Escalation email and Cliq message	Add an Escalation Log whose level matches an Escalation Rule with Notification Enabled = Yes.	Email is sent to the administrator and a message is posted to the itsupportescalations channel.	Escalation message and email were tested and showed the expected details (Section 5.7). Screenshot evidence to be attached.	Pass
TC-16	FR-06	(Negative) No matching rule or notification disabled	Add an Escalation Log with a level that has no rule; repeat with a rule set to Notification Enabled = No.	No email or Cliq message is sent; the log record is still saved.	No notification was sent when no matching active notification rule was available or notifications were disabled.	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-17	FR-07	(Negative) Edit a non-resolved ticket	Edit and save a ticket whose status is Open or In Progress.	No resolution email is sent to the customer.	No customer resolution email was sent when a non-resolved ticket was edited.	Pass
TC-18	FR-08	(Workflow) Escalation count update	Note a ticket's Escalation Count; add two Escalation Logs for it in turn.	Count increases by one per log (0, 1, 2) and Escalation Level takes the level of the latest log.	Escalation Count increased by one for each new log, and Escalation Level reflected the latest log.	Pass
TC-19	FR-09	(Positive) Filtered and grouped reports	Open Open Service Requests, Priority-Wise Requests and Agent-Wise Performance with tickets of mixed status, priority and agent.	Open Service Requests lists only tickets with status Open; the others group records by Priority and by Assigned Agent.	Reports displayed the expected records grouped and filtered by status, priority and assigned agent.	Pass
TC-20	FR-10	(Workflow) Service request lifecycle	Perform Assign to Agent, Resolve Ticket and Close Ticket on a ticket; then Reopen it.	Status moves Open, In Progress, Resolved, Closed, In Progress; first response time and resolution time are recorded in minutes.	Abstract and Section 6.1 state that transitions were tested; no per-step result was supplied.	Pending
TC-21	FR-10	(Workflow) SLA outcome on resolution	Resolve one ticket before and another after its Resolution Deadline.	SLA Status is Resolved Within SLA for the first and Resolved Breached for the second.	Tickets resolved before the deadline were marked Resolved Within SLA; tickets resolved after the deadline were marked Resolved Breached	Pass
TC-22	FR-10	(Workflow) Escalation blueprint	On an Escalation Log record perform Auto-Escalate, Escalate Further and Resolve Escalation; repeat with Re-escalate.	Stages follow Pending, Escalated, Escalated Further, Resolved; Resolution Status is updated by each transition.	Escalation Blueprint transitions completed successfully, and the expected resolution statuses were updated.	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-23	FR-11	(Role access) Support Agent menu	Log in as a Support Agent and review the menu and forms.	Agent can add Service Request and Escalation Log records; SLA Policy and Escalation Rules forms and the Dashboard are not available.	The Support Agent could access the permitted forms, while restricted components were unavailable.	Pass
TC-24	FR-11	(Role access, negative) Delete restriction	As Support Manager, Department Head and Executive Leadership, look for a Delete option on records and reports.	No Delete option is offered; only the Super Admin can delete (Section 4.6).	Delete actions were unavailable to the tested roles; deletion remained restricted to Super Admin.	Pass
TC-25	FR-11	(Role access, negative) Support Manager adds SLA Policy	Log in as Support Manager and try to add an SLA Policy.	Per Table 4.3 the action is not permitted (view only).	Do not mark Pass until the Support Manager's SLA Policy creation permission is corrected and retested.	Pass
TC-26	FR-11	(Role access) Record sharing	Create a ticket as Support Agent; then open it as Support Manager, Department Head and Executive Leadership.	Manager can view and edit; Department Head and Executive Leadership can view, as defined by the sharing rules.	Records were accessible with the expected view and edit permissions across the tested roles.	Pass

6.3 Debugging Deluge Scripts

The exported scripts run on form On Add and On Edit success events and as Blueprint transition actions. They show the following debugging and defensive practices:

- **Info statement**: DS-02 writes the calculated resolution deadline to the log with an info statement, so the value can be checked after a test record is created.
- **Null checks before use**: DS-01 checks the SLA Policy and its target; DS-03 exits when no agent is assigned; DS-04 checks that a matching rule and request exist; DS-05 checks the customer email; DS-06 checks the request record; the Resolve Ticket and Assign to Agent scripts check dates before calculating times.

- **Single-record fetches**: related records are fetched by ID or key criteria, for example service_request[ID == input.ID], so the value returned is easy to inspect.
- **Notification content**: the assignment and escalation messages include the ticket ID, title and priority, which allows the processed values to be confirmed on receipt.

Other debugging aids of Zoho Creator, such as workflow execution logs and error messages shown at submission, are described in the platform documentation [2].

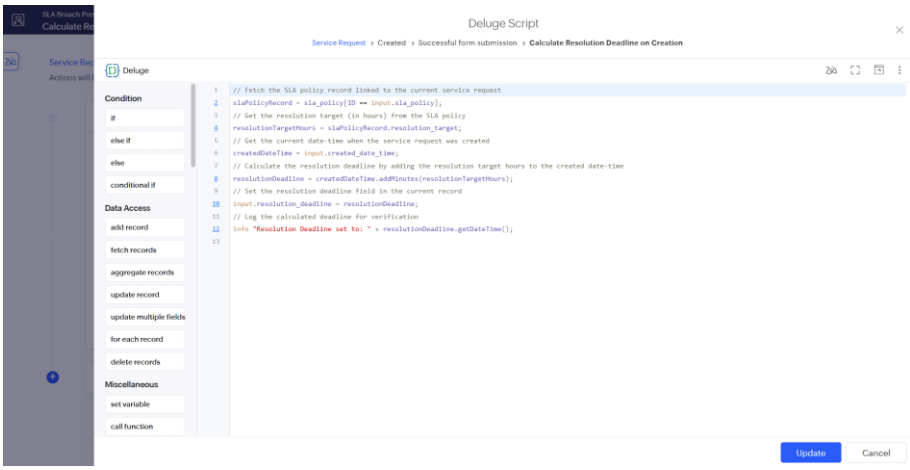


Figure 6.1: Deluge Debug Output for the Resolution Deadline Workflow

6.4 Defect Log

No defects found during test execution were supplied with the project files. Table 6.2 therefore keeps a row for observations (OBS-01 to OBS-07) found in this application. Observations are not execution results; they are recorded so that they can be confirmed or resolved.

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
OBS-01	Escalation email always goes to the administrator and is addressed to "Escalation Manager"; the rule's Notification Recipient Role is not used. The rule is matched on Escalation Level only, not Priority, Trigger Type or Active Status.	DS-04 matches escalation_level and uses zoho.adminuserid as recipient.	None. Listed in Section 9.2.	Open

Bug ID	Description	Root Cause	Fix Applied	Status
OBS-02	Table 4.3 differs from the exported profiles. Create is enabled on all four forms for Executive Leadership, on Service Request and Escalation Log for Department Head, and on SLA Policy and Escalation Rules for Support Manager.	Profile configuration differs from the design in Section 4.6.	None. Reconcile after TC-23 to TC-26.	Open
OBS-03	Table 4.2 differs from the form: Assigned Agent is not mandatory; SLA Status is mandatory; Creation Date and Time has no default value; Response Deadline, Resolution Deadline, First Response Date and Time and Resolved Date and Time are not listed.	Form changed after Table 4.2 was prepared, or the table follows the original design.	None. Update Table 4.2 or the form.	Open
OBS-04	Code 5.1 and 5.2 in Section 5.5 are abridged and differ from the exported scripts; Code 5.2 is labelled DS-02 while Table 5.2 lists the escalation-count script as DS-06. Table 5.3 omits the Increment Escalation Count workflow.	Report extracts were prepared separately from the export.	Appendix C reproduces the exported scripts with the IDs of Table 5.2.	Open
OBS-05	The response deadline (DS-01) is based on zoho.currenttime, but the resolution deadline (DS-02) uses Creation Date and Time, which has no default value. The DS-02 comment says hours although minutes are added.	The two scripts use different time bases.	None.	Open
OBS-06	Blueprint transitions named Auto-Escalate and Send Notification are performed manually and only set Resolution Status; they neither schedule escalations nor send notifications.	Each transition script holds one status assignment.	None. Names to be explained or revised.	Open
OBS-07	Application time zone in the export is India/Kolkata.	Application setting.	None.	Open

6.5 User Acceptance Testing

Table 6.3 is kept for entries from at least two users who operated the application in their assigned roles.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
User 1	Support Agent	The Service Request form provides the required fields to record and manage support tickets.	Reviewed the form configuration and retained the required fields.
User 2	Support Manager	The Escalation Log and notification features support escalation tracking.	Reviewed the escalation workflow and notification configuration.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator allows an application to be built in a Development environment and moved to Stage and Production environments by publishing, with version notes recorded at each release [2], [6]. The exported application definition (version 1.0, generated on 01-Nov-2026) identifies the application and its components but does not record which environments, release notes or rollback steps were used.

The deployment status is therefore stated only as far as the files support it:

- **Development**: forms, workflows, Blueprints, roles and sharing rules are configured, and testing used sample service requests and escalation records (Section 6.1).
- **Stage and Production**: The application was published in the Stage and Production environment and made available at: https://creatorapp.zoho.in/rathnavelu.p.2024.csbs_ra588/sla-breach-prevention-escalation-system. The published application was checked to confirm that its forms, workflows and reports were accessible.
- **Version and rollback**: the export header records version 1.0. The .ds file preserves the application definition (not the record data) and can be kept as a baseline copy.

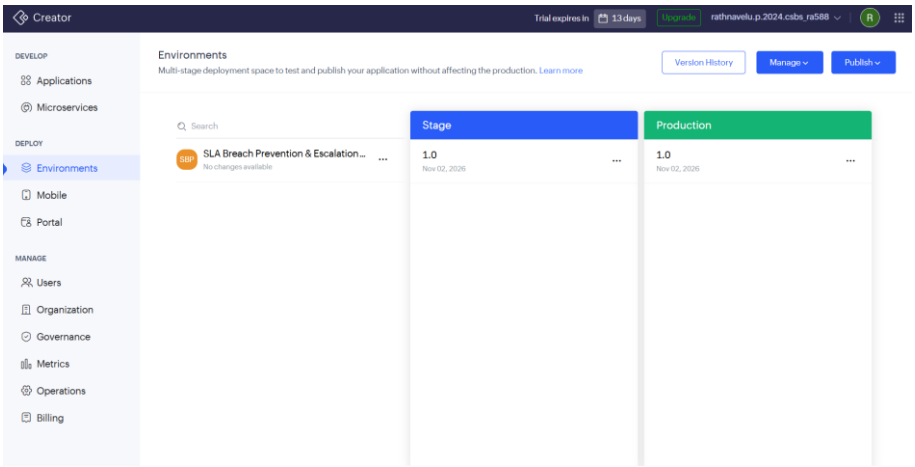


Figure 7.1: Environments and Publish to Production

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	✓
2	Sample/test data removed or isolated	✓
3	Roles, permissions and users configured in Production	✓
4	Connections authorised in Production	✓
5	Schedules and workflows enabled	✓
6	Application published and URL verified	✓
7	Version / release notes recorded	✓

7.2 Performance Optimisation

No performance measurements, such as page load times, script execution times or record-volume test results, were available. Therefore, numerical before-and-after comparisons could not be reported. The following table summarises design practices identified in the exported application that may support efficient execution. Their actual performance impact remains unmeasured.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Criteria-based single-record fetch, for example service_request[ID == input.ID]	DS-01 to DS-06	Not measured	Not measured	Not measured
No fetch statements inside loops (the scripts contain no loops)	All scripts	Not measured	Not measured	Not measured
Early exit when no agent is assigned	DS-03	Not measured	Not measured	Not measured
Report filtered by criteria ticket_status == "Open"	Open Service Requests report	Not measured	Not measured	Not measured
Aggregate counts and averages on the dashboard; add, edit, delete and duplicate actions switched off in embedded reports	Dashboard	Not measured	Not measured	Not measured

DS-01 and DS-02 each fetch the SLA Policy separately; combining them is a possible optimisation that was not applied. A measured comparison is left for future work (Section 9.2).

7.3 Security Features

Security is implemented through Zoho Creator profiles, role-based record-sharing rules and an OAuth connection for Zoho Cliq. Table 7.3 summarises the security controls identified in the exported application.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Profile-based permissions for Support Agent, Support Manager, Department Head, Executive Leadership and Administrator	Forms, reports and dashboard (Sections 4.6, 5.8)	Unauthorised access to application components
Role hierarchy (CEO, Executive Leadership, Department Head, Support Manager, Support Agent) with 8 sharing rules, Read or Read/Write	Service Request and Escalation Log records	Access to records outside a role's responsibility
Role profiles list View and Edit only for reports; no Delete	Role profiles	Accidental or unauthorised deletion
OAuth connection cliq_escalation_connection; scripts hold only the connection name	Cliq posts in DS-03 and DS-04	Credential leakage
Mandatory and unique field settings	All forms; Policy Name and Rule Name unique	Incomplete or duplicate data
Record actions disabled on reports embedded in the dashboard	Dashboard	Data changes from the dashboard
Encrypted fields, IP restrictions and two-factor authentication	Not identified in the application export	Potential exposure of sensitive information; account-level protection requires verification

The export does not establish whether encrypted fields, IP restrictions or two-factor authentication have been enabled at the organisation or account level. These settings should be confirmed separately.

The export also shows that API, PII and ePHI access are enabled in the profiles. The Cliq connection requests multiple permissions, although the scripts use channel-posting operations. These permissions should be reviewed and reduced where possible according to the principle of least privilege. Channel unique names are stored in the scripts and should be maintained carefully.

7.4 Governance and Administration

Governance practices supported by the files are as follows. Application and component link names use lower-case words joined by underscores, and workflows have descriptive display names; one workflow link name (Increment_Escalation_Coun) is truncated, although its display name is correct. The Super Admin manages users, profiles, roles, sharing rules and connections; the Department Head maintains SLA

Policies and Escalation Rules; and the Support Manager maintains escalation records (Section 3.1). The export of 01-Nov-2026, version 1.0, serves as a backup of the application definition.

Audit trail reviews and licence-management procedures have not been formally documented. These activities are recommended as part of future application governance.

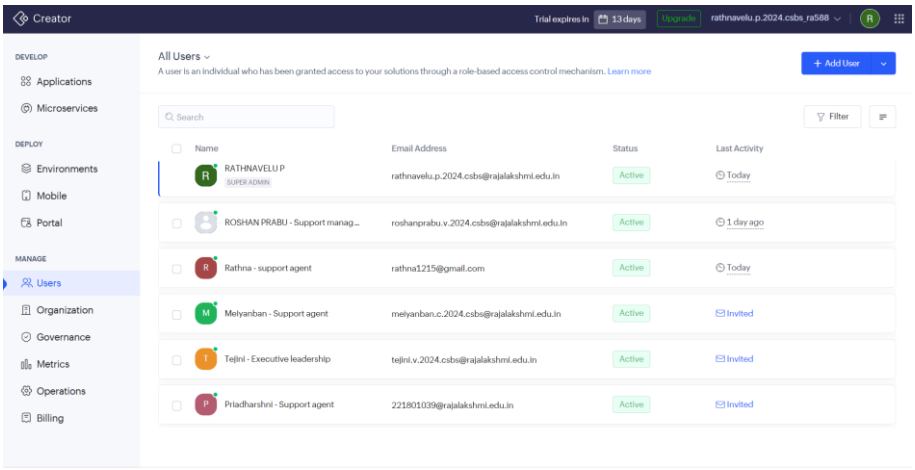


Figure 7.2: Audit Trail and Administration Settings

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

This section follows one scenario through three roles. It describes the behaviour defined by the exported configuration; Figures 8.1 to 8.3 are to be replaced with the author's screenshots of the live application. A customer reports a network outage. A Critical priority ticket is recorded, escalated and resolved as follows.

Step 1 – Ticket creation (Support Agent or Support Manager). The user opens the Service Request form, enters the details, selects the SLA Policy and the assigned agent, and submits. The workflows calculate the response and resolution deadlines, and the agent is notified by email and by a Zoho Cliq message.

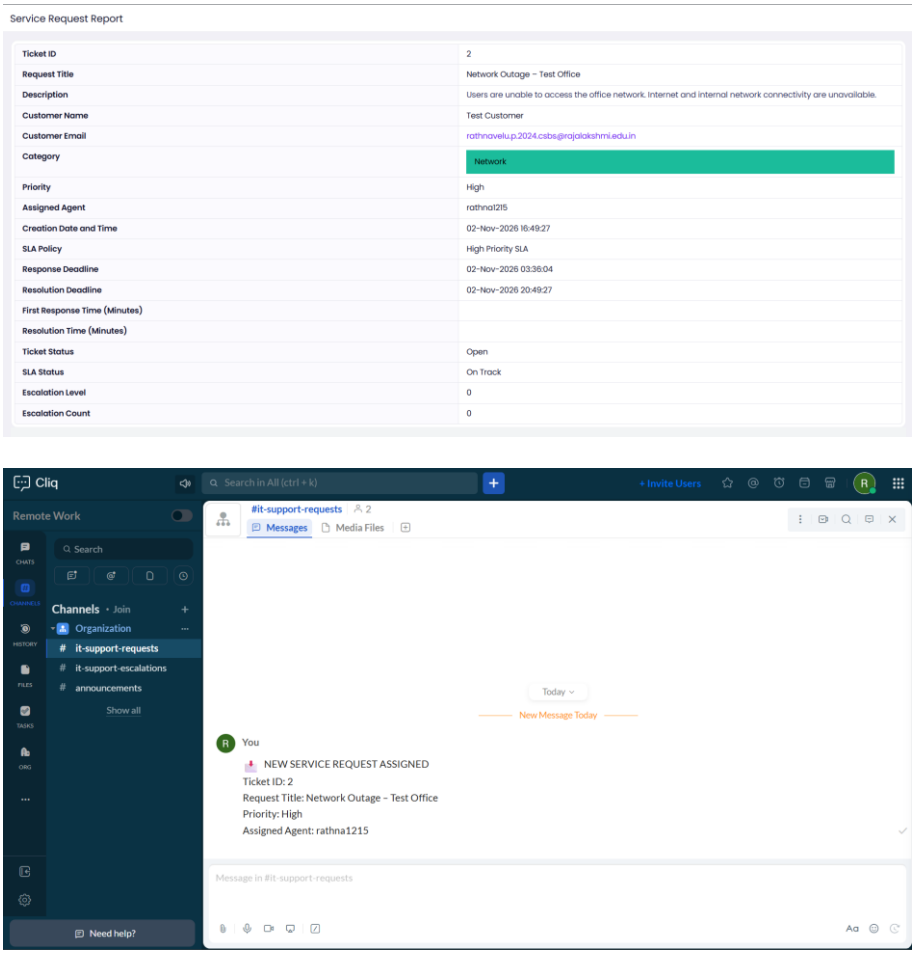


Figure 8.1: Scenario Step 1 – Service Request Created with Calculated Deadlines

Step 2 – Escalation (Support Manager). When the ticket approaches or passes a deadline, the manager adds an Escalation Log entry. The escalation email and Cliq

message are sent if a matching rule has notification enabled, and the ticket's Escalation Count and Escalation Level are updated.

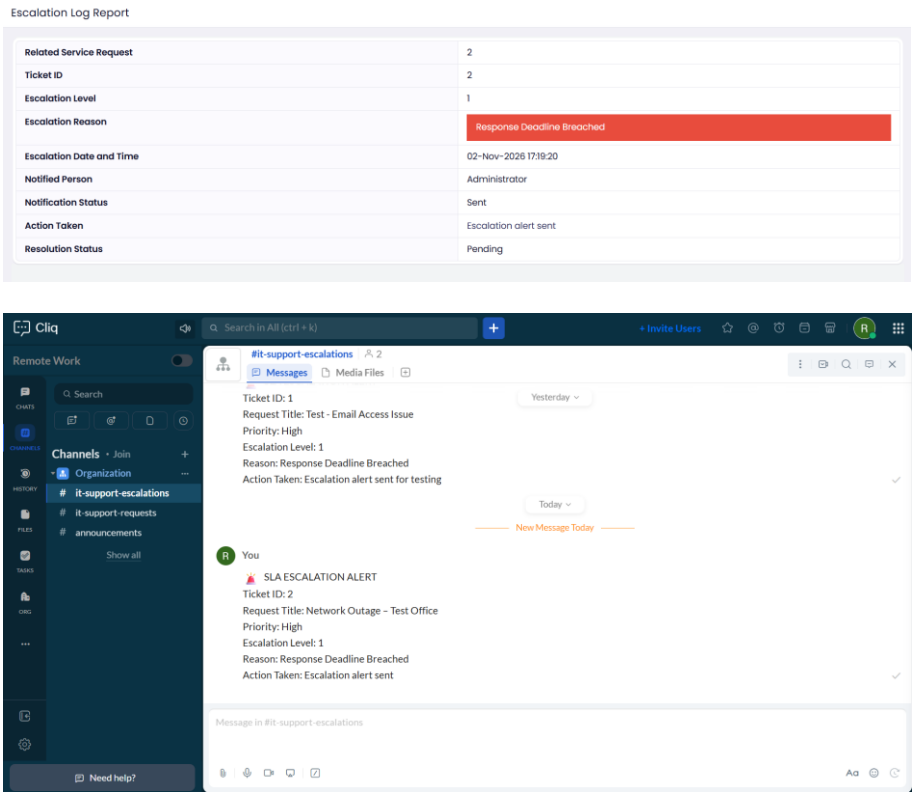


Figure 8.2: Scenario Step 2 – Escalation Recorded and Notified

Step 3 – Resolution and monitoring (Support Agent, Executive Leadership). The agent applies the Resolve Ticket transition. The resolved time, resolution time and SLA Status are recorded and the customer is emailed. The Dashboard counts then reflect the change.

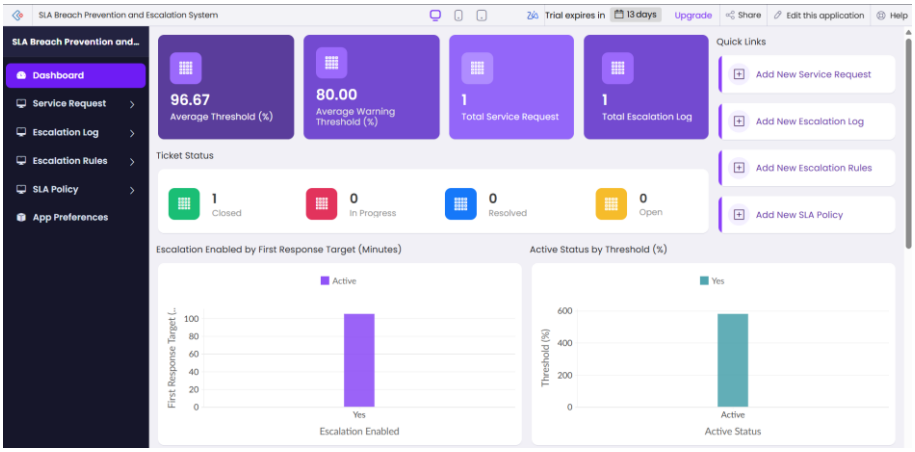


Figure 8.3: Scenario Step 3 – Ticket Resolved and Dashboard Updated

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Forms for service requests, SLA policies, escalation rules and escalation logs	Sections 4.3, 5.2; Figures 5.2, 5.3	Four forms configured. Differences from Table 4.2 are listed as OBS-03.
Calculate response and resolution deadlines	Section 5.5; DS-01, DS-02; BP-01, BP-02	Scripts configured; execution result pending (TC-08, TC-21).
Reports and a dashboard	Sections 5.3, 5.4	Eight list reports and a dashboard configured; TC-04 and TC-05 Pass.
Email and Zoho Cliq notifications	Sections 5.6, 5.7; DS-03 to DS-05	Assignment, resolution and escalation notifications tested (TC-02, TC-03, TC-15 Pass). Escalation email goes to the administrator (OBS-01).
Role-based access	Sections 4.6, 5.8, 7.3	Profiles, role hierarchy and sharing rules configured; login-based checks pending (TC-23 to TC-26); see OBS-02.
Blueprints for the request and escalation lifecycles	Section 5.6; Figures 5.6, 8.4	Two Blueprints configured; per-step results pending (TC-20 to TC-22).

Figure 8.4 shows both Blueprints as configured in the export.

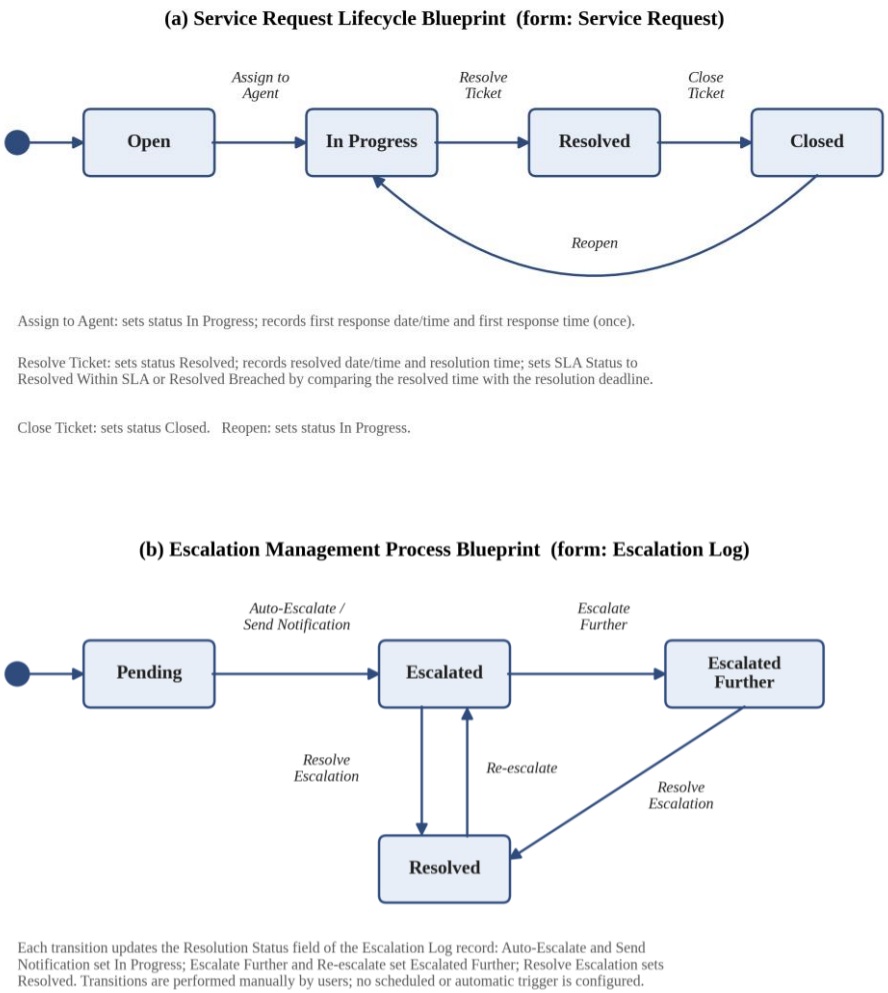


Figure 8.4: Blueprint Stages and Transitions as Configured in the Application

8.3 Comparison with the Existing System

No timing or effort measurements were taken, so the comparison in Table 8.2 is qualitative and measured improvement is not claimed.

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Time to process one request	Not measured	Not measured	Not established
Deadline calculation	Checked manually (Section 2.1)	Calculated by scripts when a ticket is created	Manual step replaced; result pending TC-08
Assignment and escalation messages	Sent manually by email or messaging	Email and Cliq messages from form events	Automated on record creation; see OBS-01
Escalation monitoring	Manual review of deadlines	Escalations logged manually; no scheduled monitoring	No improvement in detection
Report generation	Prepared manually from records	Eight reports and a dashboard	Available without manual compilation

Parameter	Existing System	Proposed Application	Improvement
Data accessibility	Records held separately; no role-based access	Central forms with profiles and sharing rules	Central, role-based; checks pending TC-23 to TC-26

8.4 Limitations of the Low-Code Approach

The following limitations are evidenced by the exported application.

- **No scheduled monitoring**: detecting approaching or missed deadlines needs a scheduled workflow, which was not implemented (Section 1.4). Workaround: escalations are logged manually and notifications fire on record creation.
- **Recipient resolution**: the rule stores a Notification Recipient Role but the script sends to the administrator. Resolving a role to users needs extra lookup logic.
- **Fixed integration details**: Cliq channel and connection names are written in the scripts, so changing them means editing the scripts.
- **Interface limits**: the dashboard is built from the platform's panels, charts and embedded reports; behaviour beyond these would need a custom widget (pro-code).
- **Platform dependence**: the .ds definition works only on Zoho Creator, so moving to another platform would require rebuilding the application.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Figure 8.5 places the parts of the application on the spectrum. Forms, reports, the dashboard, navigation, profiles, sharing rules and Blueprint design were configured without code. Deluge was needed for deadline calculation, notifications, the escalation count and the Blueprint transition actions. No pro-code extension was used.

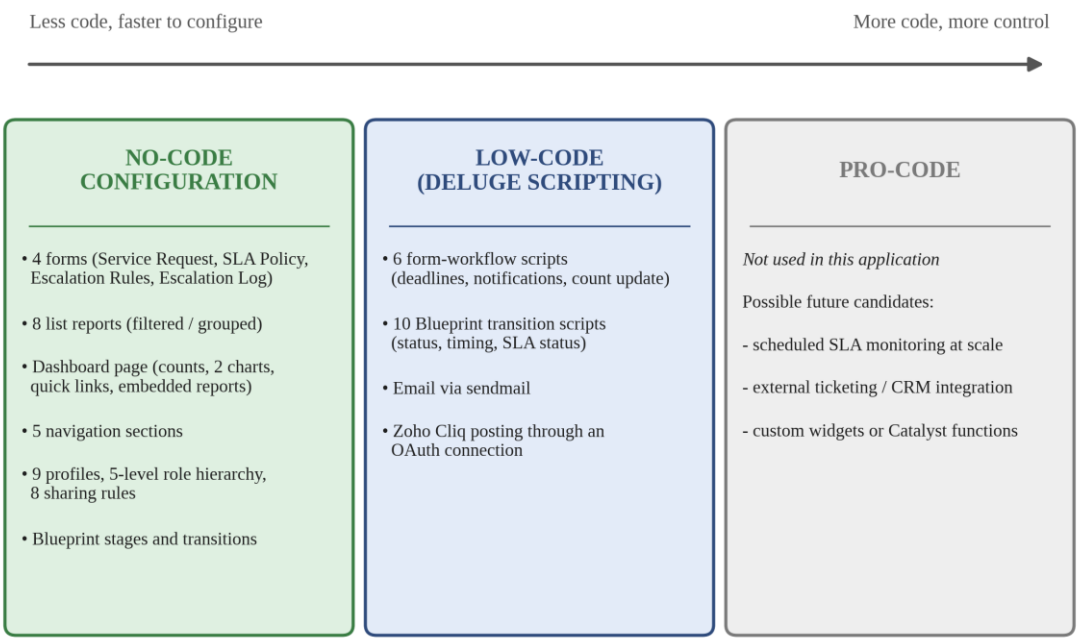


Figure 8.5: Application Components on the No-Code, Low-Code and Pro-Code Spectrum

Most of the requirements were met at the first two levels. A move to pro-code would be considered if scheduled SLA checks had to run at high volume, if the application had to exchange data with an external ticketing or CRM system, or if the interface needed behaviour the platform components cannot provide.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The SLA Breach Prevention and Escalation System was built on Zoho Creator with four forms, eight reports, a dashboard, six form-workflow scripts, two Blueprints with ten transition scripts, a Zoho Cliq connection and a role hierarchy with profiles and sharing rules. It records service requests, calculates response and resolution deadlines, notifies agents, customers and administrators, records escalations and updates escalation counts.

Testing recorded six passed cases; the remaining twenty are pending execution, and eight observations were listed for correction or confirmation. Scheduled SLA monitoring, deployment verification and measured improvements are not part of the current version.

9.2 Future Enhancements

- **Scheduled SLA monitoring**: a scheduled workflow that compares the current time with the deadlines and warning thresholds and creates Escalation Log records automatically.
- **Rule-driven escalation**: select rules by priority, trigger type, threshold and active status, and resolve the recipient role to actual users.
- **Notification refinements**: send the resolution email once only and make the notification channels configurable.
- **SLA compliance reporting**: charts based on SLA Status and escalation counts, and recorded measurements of response and resolution times.
- **Business-hours calendars**: calculate deadlines using working hours and holidays.
- **Customer portal and mobile access**: a self-service portal and verified use of the Zoho Creator mobile app.
- **AI and pro-code options**: Zia-based breach prediction and Catalyst or widget extensions for external integrations.

9.3 Learning Outcomes and Career Relevance

The project developed skills in form and data-model design, Deluge scripting, Blueprint configuration, role-based access, Zoho Cliq integration, test design and technical documentation. Comparing this report with the exported application also showed the need to verify documentation against the working system.

These skills relate to the roles of citizen developer, low-code developer, solutions consultant and automation engineer. The experience of deciding what to configure, what to script and what would need pro-code supports progression from citizen developer towards professional low-code development.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/

[3] A. Bucaioni, A. Cicchetti, and F. Ciccozzi, "Modelling in low-code development: A multi-vocal systematic review," *Software and Systems Modeling*, vol. 21, pp. 1959–1981, 2022.

[4] A. C. Bock and U. Frank, "Low-Code Platform," *Business & Information Systems Engineering*, vol. 63, pp. 733–740, 2021.

[5] D. Pinho, A. Aguiar, and V. Amaral, "What about the usability in low-code platforms: A systematic literature review," *Journal of Computer Languages*, vol. 74, Art. no. 101185, 2023.

[6] Zoho Corporation, "Zoho Creator Help – Resource Center." [Online]. Available: https://www.zoho.com/creator/help/. [Accessed: Nov. 1, 2026].

[7] UK Government, "Service Management Good Practice," Jan. 7, 2015. [Online]. Available: https://www.gov.uk/government/publications/public-services-network-psn-service-management-good-practice/service-management-good-practice. [Accessed: Nov. 1, 2026].

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Done: Y – implemented; Partial – only part of the feature is implemented; N – not implemented. Figure and table numbers refer to this report; Fig. 6.1, 7.1 and 7.2 await screenshots.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Application with 5 menu sections: Dashboard, Service Request, Escalation Log, Escalation Rules, SLA Policy	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	Y	10 types: auto number, single line, multi line, name, email, drop-down, date-time, number, percentage, radio buttons	5.2	Table 4.2; Fig. 5.2, 5.3
II	Lookup field (relationship between forms)	Y	Service Request → SLA Policy; Escalation Log → Service Request	4.2, 5.2	Table 4.1
II	Subform (parent–child data)	N	Not used	4.2, 5.2	—
II	Field validations / layout rules	Partial	Mandatory, unique and default-value settings; no validation script or layout rules	5.2	Table 4.2
II	List report with filter, grouping, custom action	Partial	Open Service Requests (filter); Priority-Wise Requests and Agent-Wise Performance (grouping); no custom action	5.3	Table 5.1; Fig. 5.4
II	Kanban / Calendar / Timeline / Map report	N	Not used	5.3	—
II	Pivot table / chart report	N	No chart or pivot report; charts are on the Dashboard	5.3	—
II	Page / dashboard built with UI Builder	Y	Dashboard page	5.4	Fig. 5.5
II	Deluge – form events (validate / success / field actions)	Partial	On Add / On Edit success events and Blueprint transition actions; no on-validate or field actions	5.5	Code 5.1–5.3; App. C
II	Deluge – functions, fetch, loops, collections	Partial	Record fetch in DS-01 to DS-06; no custom functions, loops or collections	5.5	App. C
II	Email / in-app notifications	Y	sendmail in DS-03, DS-04, DS-05 and Cliq messages in DS-03, DS-04; no in-app notification	5.5, 5.6	App. C

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Form workflow	Y	6 form workflows on Service Request and Escalation Log	5.6	Table 5.3
II	Scheduled workflow	N	Not implemented (Section 1.4)	5.6	—
II	Approval process	N	Not configured	5.6	—
II	Blueprint	Y	Service Request Lifecycle; Escalation Management Process	5.6	Fig. 5.6, 8.4
II	Batch / report workflow	N	Not used	5.6	—
II	Testing & debugging evidence	Partial	Table 6.1 (6 Pass, 20 Pending); info statement in DS-02; debug screenshot pending	6	Table 6.1; Fig. 6.1
III	Integration with a Zoho service	Y	Zoho Cliq channels itsupportrequests and itsupportescalations	5.7	Table 5.4; Code 5.3; Fig. 5.7
III	Connection / REST API call (invokeurl)	Partial	OAuth connection cliq_escalation_connection used by the Cliq task; no invokeurl call	5.7	Fig. 5.7
III	Webhook or Creator REST API	N	Not used	5.7	—
III	Users, roles and permission profiles	Y	5-level role hierarchy, profiles and 8 sharing rules	5.8	Table 4.3; Fig. 5.8
III	Record-level access criteria / customer portal	Partial	Role-based sharing rules; no criteria-based record filters or portal	5.8	Fig. 5.8
III	Zia / AI feature	N	Not implemented	5.9	—
III	Analytics (charts / Zoho Analytics)	Y	Two dashboard charts and aggregate counts; no Zoho Analytics	5.9	Fig. 5.5, 5.9
IV	Performance optimisation applied & measured	N	Design practices noted; no measurement	7.2	Table 7.2
IV	Deployment across environments	Y	Stage and production deployment	7.1	Fig. 7.1
IV	Security controls (encryption, OAuth, permissions)	Partial	Profiles, sharing rules, OAuth connection; no encrypted fields found	7.3	Table 7.3
IV	Governance (audit trail, administration)	N	No audit trail documented	7.4	Fig. 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used	5.10	—
V	Low-code limitations & spectrum analysis	Y	Sections 8.4 and 8.5	8.4, 8.5	Fig. 8.5
—	Mobile app / portal access	N	Not implemented	5.11	—

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	3 - High	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3 - High	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	2 - Medium	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2 - Medium	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	3 - High	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Service Request – Form Workflows

DS-01: Calculate Response Deadline on Creation

```
// Script ID : DS-01
// Location  : Service Request -> On Add -> On Success (workflow:
Calculate Response Deadline on Creation)
// Purpose   : Calculate the response deadline from the SLA Policy
first response target

// Fetch the current service request record
serviceRequestRecord = service_request[ID == input.ID];
// Get the SLA policy lookup record
slaPolicyRecord = serviceRequestRecord.sla_policy;
// Check if SLA policy exists
if(slaPolicyRecord != null)
{
    // Get the first response target from SLA policy
    firstResponseTarget = slaPolicyRecord.first_response_target;
    // Check if first response target is not null
    if(!firstResponseTarget.isNull())
    {
        // Get the current date and time
        currentDateTime = zoho.currenttime;
        // Calculate the response deadline by adding the first
response target hours
        responseDeadline =
currentDateTime.addMinutes(firstResponseTarget);
        // Update the response deadline field in the service request
        serviceRequestRecord.response_deadline=responseDeadline;
    }
}
```

DS-02: Calculate Resolution Deadline on Creation

```
// Script ID : DS-02
// Location  : Service Request -> On Add -> On Success (workflow:
Calculate Resolution Deadline on Creation)
// Purpose   : Calculate the resolution deadline from the SLA Policy
resolution target

// Fetch the SLA policy record linked to the current service request
slaPolicyRecord = sla_policy[ID == input.sla_policy];
// Get the resolution target (in hours) from the SLA policy
resolutionTargetHours = slaPolicyRecord.resolution_target;
// Get the current date-time when the service request was created
createdDateTime = input.created_date_time;
// Calculate the resolution deadline by adding the resolution target
hours to the created date-time
resolutionDeadline =
createdDateTime.addMinutes(resolutionTargetHours);
// Set the resolution deadline field in the current record
input.resolution_deadline = resolutionDeadline;
// Log the calculated deadline for verification
info "Resolution Deadline set to: " +
resolutionDeadline.getDateTime();
```

DS-03: Notify Agent on Ticket Assignment

```
// Script ID : DS-03
// Location  : Service Request -> On Add -> On Success (workflow:
Notify Agent on Ticket Assignment)
// Purpose   : Email and Cliq notification to the assigned agent

// Check if assigned_agent field has a valid value
if(input.assigned_agent.isNull())
{
    // No agent assigned, skip email
    return;
}
// Get the assigned agent's email address
assignedAgentEmail = input.assigned_agent.email;
// Check if email is valid
if(!assignedAgentEmail.isNull() && !assignedAgentEmail.isBlank())
{
    // Prepare email subject with ticket details
    ticketId = input.ticket_id;
    requestTitle = input.request_title;
    input.priority = input.priority;
    subject = "New Service Request Assigned: " + ticketId + " - " +
requestTitle;
    // Prepare email message
    message = "<p>Dear Agent,</p>";
    message = message.concat("<p>You have been assigned a new service
request.</p>");
    message = message.concat("<p><strong>Ticket ID:</strong> " +
ticketId + "</p>");
    message = message.concat("<p><strong>Title:</strong> " +
requestTitle + "</p>");
    message = message.concat("<p><strong>Priority:</strong> " +
input.priority + "</p>");
    message = message.concat("<p>Please review and take appropriate
action.</p>");
    // Send email notification
    sendmail
    [
        from :zoho.loginuserid
        to :assignedAgentEmail
        subject :subject
        message :message
        content type :HTML
    ]
    // Send new service request notification to Zoho Cliq
    cliqMessage = "📩 NEW SERVICE REQUEST ASSIGNED\n";
    cliqMessage = cliqMessage + "Ticket ID: " + ticketId.toString() +
"\n";
    cliqMessage = cliqMessage + "Request Title: " + requestTitle +
"\n";
    cliqMessage = cliqMessage + "Priority: " +
input.priority.toString() + "\n";
    cliqMessage = cliqMessage + "Assigned Agent: " +
input.assigned_agent.toString();

zoho.cliq.postToChannel("itsupportrequests",cliqMessage,"cliq_escalati
on_connection");
}
```

DS-05: Notify Customer on Ticket Resolution

```
// Script ID : DS-05
// Location  : Service Request -> On Edit -> On Success (workflow:
Notify Customer on Ticket Resolution)
// Purpose   : Email the customer when the ticket is Resolved

// Fetch the service request record that was just updated
serviceRequestRecord = service_request[ticket_id == input.ticket_id];
// Check if the ticket status is Resolved
if(serviceRequestRecord.ticket_status == "Resolved")
{
    // Get the customer email from the service request
    customerEmail = serviceRequestRecord.customer_email;
    // Get the customer name from the composite field
    customerName = serviceRequestRecord.customer_name.first_name;
    // Get the ticket details
    ticketTitle = serviceRequestRecord.request_title;
    input.description = serviceRequestRecord.description;
    // Check if customer email is not null or empty
    if(customerEmail.isNull() == false && customerEmail.isEmpty() ==
false)
    {
        // Construct the email message
        emailSubject = "Your Ticket " + ticketTitle + " has been
Resolved";
        emailMessage = "<p>Dear " + customerName + ",</p>";
        emailMessage = emailMessage.concat("<p>Your ticket <strong>" +
ticketTitle + "</strong> has been resolved.</p>");
        emailMessage =
emailMessage.concat("<p><strong>Details:</strong></p>");
        emailMessage = emailMessage.concat("<p>" + input.description +
"</p>");
        emailMessage = emailMessage.concat("<p>Thank you for
contacting us.</p>");
        // Send the email notification
        sendmail
        [
            from :zoho.loginuserid
            to :customerEmail
            subject :emailSubject
            message :emailMessage
            content type :HTML
        ]
    }
}
```

C.2 Escalation Log – Form Workflows

DS-04: Notify on Escalation Event

```
// Script ID : DS-04
// Location  : Escalation Log -> On Add -> On Success (workflow:
Notify on Escalation Event)
// Purpose   : Email and Cliq notification for an escalation event

// Fetch the escalation rule matching this escalation level
escalationRule = escalation_rules[escalation_level ==
input.escalation_level];
// Continue only if a matching rule exists
if(escalationRule.count() > 0)
{
```

```
    // Check whether notification is enabled
    if(escalationRule.notification_enabled == "Yes")
    {
        // Send notification to the administrator
        recipientEmail = zoho.adminuserid;
        // Fetch the related service request
        serviceRequest = service_request[ID ==
input.service_request.ID];
        // Continue only if the service request exists
        if(serviceRequest.count() > 0)
        {
            // Build the email message
            message = "<p>Dear Escalation Manager,</p>";
            message = message.concat("<p>A ticket has reached the
escalation threshold.</p>");
            message = message.concat("<p><strong>Ticket ID:</strong> "
+ serviceRequest.ticket_id + "</p>");
            message = message.concat("<p><strong>Request
Title:</strong> " + serviceRequest.request_title + "</p>");
            message = message.concat("<p><strong>Priority:</strong> "
+ serviceRequest.priority + "</p>");
            message = message.concat("<p><strong>Escalation
Level:</strong> " + input.escalation_level.toString() + "</p>");
            message = message.concat("<p><strong>Escalation
Reason:</strong> " + input.escalation_reason + "</p>");
            message = message.concat("<p><strong>Action
Taken:</strong> " + input.action_taken + "</p>");
            message = message.concat("<p>Please review and take
necessary action.</p>");
            // Send the email notification
            sendmail
            [
                from :zoho.adminuserid
                to :recipientEmail
                subject :"Escalation Alert: Ticket " +
serviceRequest.ticket_id
                message :message
                content type :HTML
            ]
            // Send escalation notification to Zoho Cliq
            cliqMessage = "🚨 SLA ESCALATION ALERT\n";
            cliqMessage = cliqMessage + "Ticket ID: " +
serviceRequest.ticket_id.toString() + "\n";
            cliqMessage = cliqMessage + "Request Title: " +
serviceRequest.request_title + "\n";
            cliqMessage = cliqMessage + "Priority: " +
serviceRequest.priority + "\n";
            cliqMessage = cliqMessage + "Escalation Level: " +
input.escalation_level.toString() + "\n";
            cliqMessage = cliqMessage + "Reason: " +
input.escalation_reason + "\n";
            cliqMessage = cliqMessage + "Action Taken: " +
input.action_taken;

zoho.cliq.postToChannel("itsupportescalations",cliqMessage,"cliq_escal
ation_connection");
        }
    }
}
```

DS-06: Increment Escalation Count

```
// Script ID : DS-06
// Location  : Escalation Log -> On Add -> On Success (workflow:
Increment Escalation Count)
// Purpose   : Increment the escalation count and update the
escalation level of the service request

serviceRequestRec = service_request[ID == input.service_request.ID];
if(serviceRequestRec != null)
{
    currentCount = ifnull(serviceRequestRec.escalation_count,0);
    serviceRequestRec.escalation_count=currentCount + 1;
    serviceRequestRec.escalation_level=input.escalation_level;
}
```

C.3 Service Request Lifecycle – Blueprint Transition Scripts

BP-01: Assign to Agent (Open to In Progress)

```
// Script ID : BP-01
// Location  : Service Request -> Blueprint "Service Request
Lifecycle" -> transition "Assign to Agent" (Open to In Progress),
After actions
// Purpose   : Set status In Progress and record first response
date/time and time

// Update ticket status
input.ticket_status = "In Progress";
// Record first response only once
if(input.first_response_date_and_time == null)
{
    input.first_response_date_and_time = zoho.currenttime;
    if(input.created_date_time != null)
    {
        elapsedMilliseconds =
input.first_response_date_and_time.toLong() -
input.created_date_time.toLong();
        input.first_response_time = (elapsedMilliseconds /
60000).toLong();
    }
}
```

BP-02: Resolve Ticket (In Progress to Resolved)

```
// Script ID : BP-02
// Location  : Service Request -> Blueprint "Service Request
Lifecycle" -> transition "Resolve Ticket" (In Progress to Resolved),
After actions
// Purpose   : Set status Resolved; record resolved date/time,
resolution time and SLA status

// Update ticket status
input.ticket_status = "Resolved";
// Record resolution time
input.resolved_date_and_time = zoho.currenttime;
// Calculate total resolution time in minutes
if(input.created_date_time != null)
{
    elapsedMilliseconds = input.resolved_date_and_time.toLong() -
input.created_date_time.toLong();
```

```
    input.resolution_time = (elapsedMilliseconds / 60000).toLong();
}
// Update SLA status
if(input.resolution_deadline != null)
{
    if(input.resolved_date_and_time <= input.resolution_deadline)
    {
        input.sla_status = "Resolved Within SLA";
    }
    else
    {
        input.sla_status = "Resolved Breached";
    }
}
```

BP-03: Close Ticket (Resolved to Closed)

```
// Script ID : BP-03
// Location  : Service Request -> Blueprint "Service Request
Lifecycle" -> transition "Close Ticket" (Resolved to Closed), After
actions
// Purpose   : Set status Closed

// Update ticket status
input.ticket_status = "Closed";
```

BP-04: Reopen (Closed to In Progress)

```
// Script ID : BP-04
// Location  : Service Request -> Blueprint "Service Request
Lifecycle" -> transition "Reopen" (Closed to In Progress), After
actions
// Purpose   : Set status In Progress

// Update ticket status
input.ticket_status = "In Progress";
```

C.4 Escalation Management Process – Blueprint Transition Scripts

BP-05: Auto-Escalate (Pending to Escalated)

```
// Script ID : BP-05
// Location  : Escalation Log -> Blueprint "Escalation Management
Process" -> transition "Auto-Escalate" (Pending to Escalated), After
actions
// Purpose   : Set Resolution Status to In Progress

input.resolution_status = "In Progress";
```

BP-06: Send Notification (Pending to Escalated)

```
// Script ID : BP-06
// Location  : Escalation Log -> Blueprint "Escalation Management
Process" -> transition "Send Notification" (Pending to Escalated),
After actions
// Purpose   : Set Resolution Status to In Progress
```

```
input.resolution_status = "In Progress";
```

BP-07: Escalate Further (Escalated to Escalated Further)

```
// Script ID : BP-07
// Location  : Escalation Log -> Blueprint "Escalation Management
Process" -> transition "Escalate Further" (Escalated to Escalated
Further), After actions
// Purpose   : Set Resolution Status to Escalated Further

input.resolution_status = "Escalated Further";
```

BP-08: Resolve Escalation (Escalated to Resolved)

```
// Script ID : BP-08
// Location  : Escalation Log -> Blueprint "Escalation Management
Process" -> transition "Resolve Escalation" (Escalated to Resolved),
After actions
// Purpose   : Set Resolution Status to Resolved

input.resolution_status = "Resolved";
```

BP-09: Resolve Escalation (Escalated Further to Resolved)

```
// Script ID : BP-09
// Location  : Escalation Log -> Blueprint "Escalation Management
Process" -> transition "Resolve Escalation" (Escalated Further to
Resolved), After actions
// Purpose   : Set Resolution Status to Resolved

input.resolution_status = "Resolved";
```

BP-10: Re-escalate (Resolved to Escalated)

```
// Script ID : BP-10
// Location  : Escalation Log -> Blueprint "Escalation Management
Process" -> transition "Re-escalate" (Resolved to Escalated), After
actions
// Purpose   : Set Resolution Status to Escalated Further

input.resolution_status = "Escalated Further";
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

1. Open Zoho Creator in Google Chrome or Microsoft Edge and sign in with the user ID and password given by the Super Admin.
2. Open the application SLA Breach Prevention and Escalation System. https://creatorapp.zoho.in/rathnavelu.p.2024.csbs_ra588/sla-breach-prevention-and-escalation-system
3. Use the left menu. It offers Dashboard, Service Request, Escalation Log, Escalation Rules and SLA Policy; components not permitted for your role may not appear.

D.2 Instructions for Support Agent

1. Open Service Request and fill in the form: Request Title, Description, Customer Name, Customer Email, Category, Priority, Creation Date and Time, SLA Policy, Assigned Agent, Ticket Status and SLA Status. Click Submit.
2. After submission the response and resolution deadlines are calculated and the assigned agent is notified by email and Zoho Cliq.
3. To work on a ticket, open it from Service Request Report or Open Service Requests and apply the Blueprint transition Assign to Agent. When the issue is fixed, apply Resolve Ticket; the customer is then emailed.
4. To report an escalation, open Escalation Log and add a record for the ticket.
5. View Priority-Wise Requests, Agent-Wise Performance, Escalation History, SLA Policy Report and Escalation Rules Report for reference. Agents cannot add SLA Policies or Escalation Rules and have no Dashboard access.

D.3 Instructions for Support Manager

1. Open Dashboard to see the request and escalation totals, ticket-status counts, charts and quick links.
2. Review tickets in Service Request Report, Open Service Requests, Priority-Wise Requests and Agent-Wise Performance; edit tickets where needed.
3. Add an Escalation Log for a ticket that is near or past its deadline: choose the Related Service Request, Escalation Level, Reason, Date and Time, Notification Status and Resolution Status.

4. Move the escalation through the Escalation Management Blueprint (for example Escalate Further or Resolve Escalation).
5. View SLA Policy Report and Escalation Rules Report. Changes to policies and rules are made by the Department Head.

D.4 Instructions for Department Head

1. Open SLA Policy and add a policy: Policy Name, Priority, First Response Target (minutes), Resolution Target (minutes), Warning Threshold (default 80%), Escalation Enabled and Active Status.
2. Open Escalation Rules and add a rule: Rule Name, Priority, Trigger Type, Threshold (%), Escalation Level, Notification Recipient Role, Notification Enabled and Active Status. The notification script matches a rule by Escalation Level.
3. Use SLA Policy Report and Escalation Rules Report to edit existing entries.
4. Open Dashboard and the priority-wise and escalation reports to review department performance.

D.5 Instructions for Executive Leadership

1. Open Dashboard for overall request and escalation totals and ticket-status counts.
2. Open Service Request Report, Escalation Log Report and Escalation History to review service performance and escalations.
3. Edit records only where required; the role has View and Edit permission.

D.6 Instructions for Super Admin

1. Add users and assign each a profile and role (Support Agent, Support Manager, Department Head, Executive Leadership).
2. Review profiles and sharing rules when permissions change.
3. Check that the connection cliq_escalation_connection is authorised so that Zoho Cliq messages are posted.
4. Delete records only when necessary; deletion is restricted to the Super Admin.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	RATHNA VELU P	Register No.	241401078
Project Title	SLA Breach Prevention and Escalation System	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.

Level	% of Max. Marks	Descriptor
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	RATHNA VELU P	Register No.	241401078
Department / Section	Computer Science and Business Systems	Date of Viva	
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____